Linux System Administration (E1ITA307)

Linux Infrastructure Deployment Project Report

On

CENTRALIZED BACKUP AND RECOVERY SERVER

Submitted by

NANDANA N (2411021061223)

PUJITHA S (2411021061630)

ABINAYATHRI A (2411021061395)

K B SANJANA (2411021061602)

In partial fulfillment of the award of the degree of

BACHELOR OF TECHNOLOGY

In

Information Technology

Under the Supervision of

Prof. Ananthanagu U

ALLIANCE SCHOOL OF ADVANCED COMPUTING

ALLIANCE UNIVERSITY

BENGALURU

Oct 2026

DEPARTMENT OF INFORMATION TECHNOLOGY

ALLIANCE SCHOOL OF ADVANCED COMPUTING

CERTIFICATE

This is to certify that the Project work entitled **"Centralized Backup and Recovery Server"** submitted by Abinayathri A [2411021061395], Nandana N [2411021061223], Pujitha S [2411021061630] and K B Sanjana [2411021061602] in partial fulfillment of the requirements for the award of the degree of Bachelor of Technology in Information Technology-IT - DA of Alliance University, is a bonafide work during the academic year 2026-2027.

Prof. Ananthanagu U	Prof. Ananthanagu U	Dr. Shekhar R
Course Handling Faculty	Course Coordinator	Professor & HoD

Table of Contents

1. Introduction **4**
1.1 Background of the Project 4
1.2 Purpose of the Project 5
1.3 Overview of the System 5
1.4 Objectives of the Project 5
1.5 Problem Statement 6
1.6 Scope of the Project 6
1.7 Significance of the Project 7
2. Methodology **8**
2.1 Introduction to Methodology 8
2.2 System Requirements 8
2.3 Tools and Technologies Used 10
2.4 Approach / Development Process 11
3. Linux System Administration – Design and Implementation **13**
3.1 System Architecture 13
3.2 Linux Environment Setup 14
3.2.1 Backend Processing 15
3.2.2 Frontend and Menu Interface 19
3.2.3 Full Backup Implementation 25
3.2.4 Incremental Backup Implementation 27
3.2.5 Listing and Managing Backups 29
3.2.6 Scheduled Backup Management Using Cron 30
3.2.7 Executing Backup Operations 32
3.2.8 Viewing Scheduled Backup Logs 34
3.2.9 Application Log Management 36
3.2.10 Data Restoration 38
3.2.11 Recovery Testing 42
3.2.12 Input Validation and Error Handling 44
4. Results and Testing **47**
4.1 Implementation Results 47
4.2 Testing and Validation 48
5. Conclusion and Future Scope **50**
5.1 Conclusion 50
5.2 Future Scope 50
References **52**
Appendices **53**
Appendix A: Output Screenshot Index 53
Appendix B: Main Menu Options of the Dashboard 56

1. INTRODUCTION

Linux is a widely used open-source operating system that provides a stable and flexible environment for managing files, applications, users, and system resources. It is commonly used in servers, cloud platforms, educational institutions, and business environments. Linux provides several command-line utilities and shell scripting features that help administrators automate routine system administration tasks.

One important responsibility of system administration is protecting data from accidental deletion, system failures, software errors, and other unexpected situations. Data is an essential resource for individuals and organizations. Losing important files can interrupt daily activities and may result in additional time and effort being required to recreate the lost information.

The project titled **"Centralized Backup and Recovery Server"** focuses on providing a structured approach to backing up files and restoring them when required. The project uses Linux commands and scripting concepts to organize backup operations, maintain backup records, and support recovery testing.

The system is designed to support full backups, incremental backups, backup scheduling, backup listing, restoration, logging, and input validation. A full backup stores a complete copy of the selected data, while an incremental backup stores changes made since an earlier backup, according to the implemented backup procedure. Scheduled backups help reduce the need for users to start every backup manually.

Logging is another important part of the project. Logs provide information about backup activities and help users review operations performed by the system. Recovery testing is used to check whether backed-up data can be restored successfully. These activities help demonstrate the importance of maintaining backup copies and verifying their usability.

The project brings together Linux system administration concepts, shell scripting, file management, scheduling, and recovery procedures in a practical application. It also provides an opportunity to understand how routine administrative tasks can be organized and automated using Linux tools.

1.1 BACKGROUND OF THE PROJECT

In a computer system, files may contain academic documents, project source code, configuration files, reports, and other important information. If these files are lost or damaged, users may need to recreate them manually. A backup system helps reduce this risk by keeping additional copies of selected data.

Manual backup methods can be inconvenient because users must remember when to create copies, where to store them, and how to restore them later. A systematic backup process can make these activities easier to manage.

Linux provides commands and utilities for copying files, creating archives, checking directories, scheduling jobs, and recording command output. Shell scripts can combine these commands into a repeatable workflow. Cron can be used to schedule commands or scripts to run at specified times when configured correctly.

The Centralized Backup and Recovery Server project applies these concepts to create an organized backup and restoration workflow. It provides operations for performing backups, viewing available backup records, restoring selected data, checking logs, and testing recovery.

1.2 PURPOSE OF THE PROJECT

The main purpose of this project is to demonstrate how Linux tools and shell scripting can be used to organize backup and recovery operations. The project aims to make backup activities easier to manage, improve the traceability of operations through logs, and provide a way to test the restoration of backed-up files.

The project also supports practical learning of Linux commands, Bash scripting, scheduling, error handling, and file management.

1.3 OVERVIEW OF THE SYSTEM

The system organizes backup and recovery activities through the available project interface and supporting scripts. Users can select the operations provided by the implementation, such as running a backup, listing available backups, viewing logs, restoring data, and performing recovery tests.

The backup process stores copies of selected data according to the implemented backup method. Scheduled execution can be configured through Cron. Log records help users inspect previous activities, while restoration and recovery testing help verify whether the stored backup data can be used.

The actual behavior of each operation depends on the scripts, paths, and configuration used in the project implementation.

1.4 OBJECTIVES OF THE PROJECT

- Provide a menu-driven interface for backup and recovery operations.
- Allow users to select full backup and incremental backup tasks.
- Display available backup files and their details.
- Accept a restore destination from the user.
- Run a recovery test and display its result.
- Maintain and display log records for troubleshooting.
- Handle invalid menu choices and report operation errors.

- Demonstrate Bash control structures and standard Linux commands.

1.5 PROBLEM STATEMENT

In many Linux environments, backup operations are performed manually or through separate commands and scripts. This can make it difficult to maintain a consistent backup schedule, organize backup files, monitor completed operations, and verify whether stored data can be recovered successfully.

The problem addressed by this project is to design a Linux-based centralized backup and recovery system that supports full backups, incremental backups, scheduled backup operations, log monitoring, and recovery testing. The system uses Bash shell scripting and Linux commands to organize these activities through a menu-driven interface.

The project aims to provide a convenient way to initiate backup operations, view available backups, manage scheduling-related tasks, inspect logs, and test data restoration. It also aims to handle invalid user input and report errors encountered during operations.

1.6 SCOPE OF THE PROJECT

The scope of the Centralized Backup and Recovery Server project includes the following activities:

- Creating full backups of the configured source data.
- Creating incremental backups using the method implemented in the project.
- Listing available backup files.
- Providing a menu for scheduling-related operations using Cron.
- Running backup operations manually when required.
- Displaying scheduled backup logs and application logs.
- Restoring selected backup data to a destination.
- Testing whether backed-up data can be recovered.
- Handling invalid menu selections and displaying appropriate messages.

The project focuses on Linux administration and Bash shell scripting. Its actual capabilities depend on the scripts, directories, permissions, and scheduling configuration used in the implementation. Features such as remote replication, encryption, and automated notifications are outside the stated scope unless they have been implemented separately.

1.7 SIGNIFICANCE OF THE PROJECT

The project demonstrates how routine backup activities can be organized into a repeatable workflow. A centralized menu allows users to access the available operations without having to remember every command individually.

Full and incremental backups provide different approaches to storing data copies. Scheduling can help make backup execution more consistent, while logs provide information for troubleshooting. Recovery testing is particularly important because it checks whether stored backup data can actually be used.

The project also provides practical experience with Linux commands, Bash scripting, file handling,

Cron scheduling, error handling, and data recovery procedures.

2. METHODOLOGY

2.1 INTRODUCTION TO METHODOLOGY

Methodology describes the systematic approach followed to design, develop, implement, and test the Centralized Backup and Recovery Server. It explains the steps involved in developing the project using Bash shell scripting and Linux commands.

The main objective of the methodology is to organize backup and recovery activities into a structured workflow. The project includes full backup, incremental backup, backup listing, scheduling-related operations, log monitoring, data restoration, recovery testing, and input validation.

The development process begins with identifying the requirements of the backup system. The required Linux environment and utilities are then considered, followed by the development of scripts and the menu-driven interface. Each operation is designed to perform a specific task and display its result to the user.

After implementation, the project is tested using the available operations. Full and incremental backup functions are checked, backup files are listed, scheduling-related features and logs are inspected, and restoration and recovery testing are performed. Invalid input is also tested to observe how the system responds to unsupported choices.

The methodology emphasizes modular development, functional testing, safe recovery procedures, and clear output messages. These practices help make the project easier to understand, demonstrate, and maintain.

The following sections describe the requirements, tools, development process, and testing approach used for the project.

2.2 SYSTEM REQUIREMENTS

System requirements describe the hardware and software resources required to develop, execute, and test the Centralized Backup and Recovery Server. Since the project is implemented using Bash shell scripting and Linux commands, it requires a Linux environment with the necessary command-line utilities.

The system must provide sufficient storage space for source files, backup archives, restored data, and log files. It must also provide the permissions required to execute scripts, read source data, and write backup files.

HARDWARE REQUIREMENTS

The following hardware resources are recommended for developing and demonstrating the project:

1. **Processor:** A computer with a processor capable of running a Linux operating system.
2. **RAM:** Sufficient memory to run the operating system, terminal, and backup utilities.
3. **Storage:** Adequate disk space for source data, full backups, incremental backups, recovery-test files, and logs.
4. **Display:** A monitor or laptop display for viewing the terminal interface and project outputs.
5. **Keyboard:** Required for entering menu choices and executing commands.
6. **Additional Storage (Optional):** An external drive or separate storage volume may be used for backup storage if configured in the project.

The actual storage requirement depends on the size of the source data and the number of backup archives maintained.

SOFTWARE REQUIREMENTS

The following software components are required or relevant to the project:

1. **Linux Operating System:** Provides the environment for executing the scripts and Linux commands.
2. **Bash Shell:** Executes the shell scripts and supports variables, conditions, loops, and functions.
3. **Linux Command-Line Utilities:** Used for file handling, directory management, archive operations, and log inspection.
4. **Archive Utility:** A utility such as tar may be used to create, list, or extract backup archives if it is part of the implementation.
5. **Cron:** Used to schedule recurring backup operations when the Cron service is configured and running.
6. **Text Editor:** Used to create and modify the Bash scripts and configuration files.
7. **Terminal:** Provides the interface for executing the application and observing its output.

STORAGE REQUIREMENTS

Storage is an important requirement because the project creates backup copies and may also generate restored test files.

The backup destination must have sufficient free space for the archives produced by the application. The recovery-test destination should also have enough space to extract the selected backup data without overwriting the original files.

Log files require additional storage, particularly when scheduled jobs run frequently. The backup directory and log directory should be organized so that their contents can be inspected easily.

USER AND PERMISSION REQUIREMENTS

The user executing the project must have permission to run the shell scripts, access the source data, create backup files, and read the relevant logs.

If scheduled jobs are used, the account running those jobs must have the required permissions and access to the script paths. Incorrect permissions or unavailable directories may cause backup or recovery operations to fail.

The project should use appropriate file permissions and a safe destination for recovery testing to reduce the risk of accidental data loss.

2.3 TOOLS AND TECHNOLOGIES USED

The Centralized Backup and Recovery Server project uses Linux system administration tools, Bash shell scripting, and command-line utilities to manage backup and recovery operations. These technologies provide the facilities required to process user input, manage files, execute backup operations, schedule tasks, and display results.

The project combines these tools into an organized workflow that allows users to access backup and recovery functions through the available terminal interface.

LINUX OPERATING SYSTEM

Linux provides the execution environment for the Centralized Backup and Recovery Server. It supports file management, command execution, access permissions, and scheduled tasks. These facilities are used to manage source files, create backup copies, and execute the shell scripts required by the project.

BASH SHELL SCRIPTING

Bash is the scripting language used to control the application's execution flow. It supports variables, conditional statements, loops, functions, and command execution. In this project, Bash connects the menu options with their corresponding operations and displays messages based on the results.

LINUX COMMAND-LINE UTILITIES

Linux command-line utilities are used to perform file and directory operations, inspect stored data, and manage backup-related activities. The specific commands used in the project will be described in the implementation chapter according to the submitted source code.

CRON SCHEDULER

Cron is a time-based scheduling facility that can execute commands or scripts automatically at configured times. It is relevant to the scheduled backup functionality of the project. The actual schedule and execution behavior should match the Cron configuration used in the implementation.

ARCHIVE MANAGEMENT

Archive utilities are used to store collections of files in a manageable format and extract them during restoration. The archive utility and options described in the final report should match those used by the project scripts.

LOG MANAGEMENT

Log files provide records of application activities and scheduled operations. They help users review previous activities and investigate errors. The project provides options for viewing application logs and scheduled logs.

TERMINAL INTERFACE AND TEXT EDITOR

The Linux terminal allows users to enter menu choices, execute operations, and view output messages. A text editor is used to create and modify the shell scripts and configuration files.

These technologies work together to provide the foundation for backup creation, backup listing, scheduling, logging, restoration, and recovery testing.

2.4 APPROACH / DEVELOPMENT PROCESS

The Centralized Backup and Recovery Server was developed in a series of steps. Each stage focused on implementing and checking a specific backup or recovery function.

Step 1: Linux Environment Setup

The Linux environment was prepared for developing and executing the project. The required project files and scripts were organized in the project directory.

Step 2: Menu Development

A menu-driven interface was prepared to allow users to select the required backup and recovery operations. The menu provides options for performing backups, viewing backup records and logs, restoring data, and testing recovery.

Step 3: Full Backup Implementation

The full backup function was implemented to create a backup copy of the selected data. The output was checked to verify the result of the operation.

Step 4: Incremental Backup Implementation

The incremental backup function was developed to back up changes according to the method used in the project. The terminal output was checked after execution.

Step 5: Backup Listing and Execution

Options were included to view available backups and run a backup operation through the interface. These functions help users manage backup activities.

Step 6: Logging and Scheduling

Options were provided to view scheduled backup logs and application logs. The scheduling configuration and execution results were checked where applicable.

Step 7: Data Restoration

The restoration function was implemented to recover data from an available backup. The restoration output was observed to check whether the operation completed as expected.

Step 8: Recovery Testing

A recovery test was performed to examine the restoration process. The result was recorded as part of the project testing.

Step 9: Input Validation

Input validation was included to handle invalid menu selections and guide users toward valid options.

Step 10: Testing and Documentation

The implemented functions were executed and their terminal outputs were captured. The outputs were organized in the implementation and testing chapters to document the project's behaviour.

3. LINUX SYSTEM ADMINISTRATION – DESIGN AND IMPLEMENTATION

3.1 SYSTEM ARCHITECTURE

The Backup & Recovery Server is designed to automate the process of creating full and incremental backups, verifying backup integrity, restoring data, and testing disaster recovery on a Linux system. It uses Bash scripts, configuration files, scheduled tasks, log files, and a Python-based graphical user interface to manage backup and recovery operations.

The system runs on Ubuntu Linux. The Python Tkinter dashboard provides a user-friendly interface for performing backup operations, listing available backups, verifying checksums, viewing logs, managing scheduled tasks, and restoring data. The Bash scripts handle backup creation, data restoration, and recovery testing. The Cron scheduler automatically runs full backups every Sunday and incremental backups from Monday to Saturday at 2:00 AM. Backup archives are stored in designated directories, and SHA-256 checksums are used to verify their integrity. The restored data can be compared with the original source data to confirm successful recovery.(in figure 3.1)

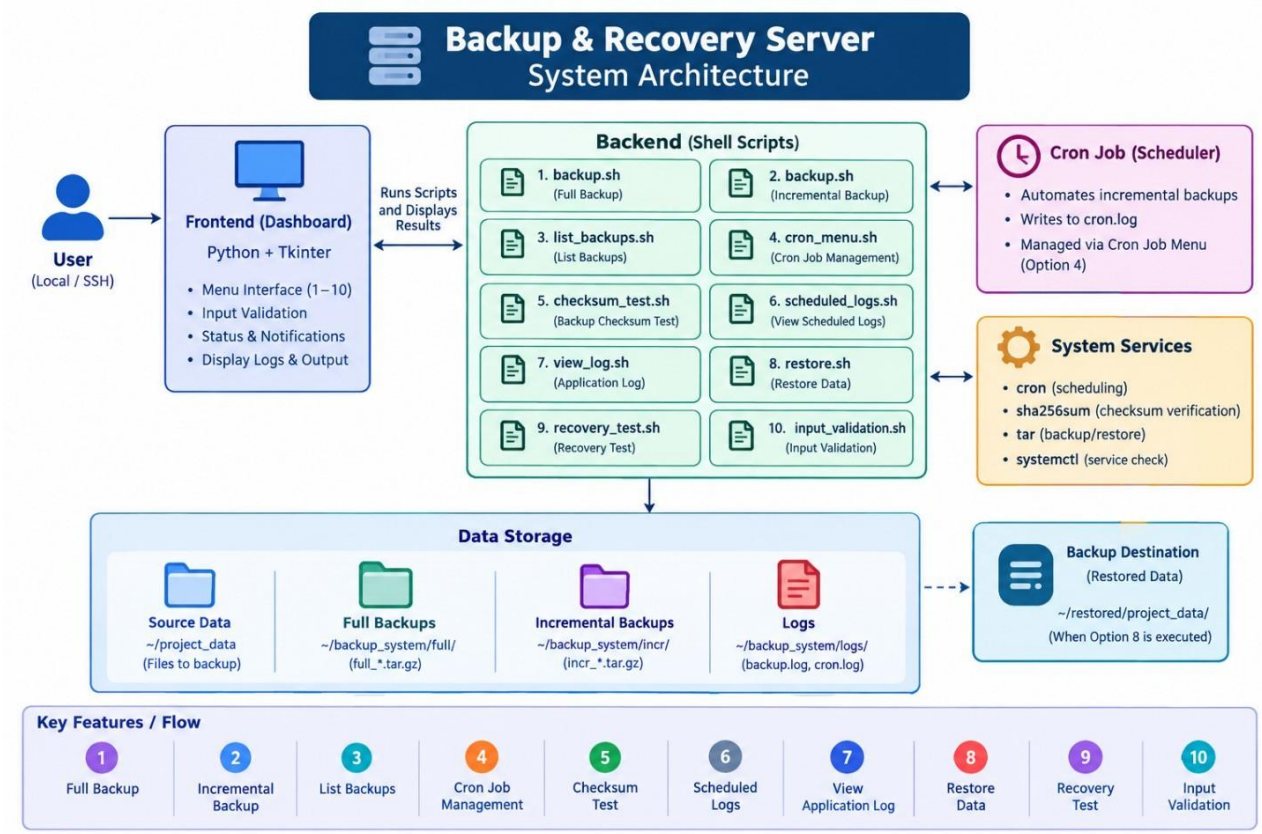


Figure 3.1 illustrates the system architecture of the Backup & Recovery Server, showing the relationships between the user interface, Bash scripts, Cron scheduler, system utilities, source data, backup storage, log files, and recovery destination.

COMPONENTS AND THEIR ROLES

1. Client/User: The user interacts with the Backup & Recovery Server through the Python Tkinter graphical dashboard. The user can initiate full and incremental backups, view available backups, check backup integrity, review logs, manage scheduled tasks, restore data, and perform recovery tests.

2. Linux Server: Ubuntu Linux provides the operating environment for the backup and recovery system. It supports Bash script execution, file management, backup processing, scheduled tasks, and system utilities.

3. Source Data and Backup Storage: The source files are stored in the `~/project_data` directory. Full backup archives are stored in `~/backup_system/full/`, while incremental backup archives are stored in `~/backup_system/incr/`. These archives help protect project data against accidental loss.

4. Configuration Module: The `config.sh` script defines the main directories used by the system, including the source data, backup locations, log file, and incremental backup snapshot. It also provides a logging function to record system activities.

5. Backup and Recovery Scripts: The `backup.sh` script creates full and incremental backups. The `restore.sh` script restores data from the latest full backup and applies subsequent incremental backups. The `recovery_test.sh` script supports testing whether the recovery process works correctly.

6. Scheduling and Automation: The Cron service automatically executes scheduled backup operations. A full backup is scheduled for every Sunday at 2:00 AM, while incremental backups are scheduled from Monday to Saturday at 2:00 AM. This reduces the need for manual backup execution.

7. Integrity Verification and Logging: The system generates SHA-256 checksum files to verify the integrity of backup archives. Backup activities, errors, and restoration operations are recorded in `~/backup_system/logs/backup.log`. These logs help the user review system operations and identify problems.

8. Graphical Dashboard and Recovery Destination: The Python Tkinter dashboard provides a user-friendly interface for accessing backup and recovery functions. Restored files can be placed in a specified destination directory, such as `~/restored/`. The recovered data can then be compared with the original source data to verify the restoration process.

3.2 LINUX ENVIRONMENT SETUP

Linux provides a suitable environment for developing and executing system administration projects. The Centralized Backup and Recovery Server is designed to use Linux commands and Bash shell scripting to perform backup and recovery operations. The Linux terminal provides an interface for executing commands, managing files and directories, handling backup operations, and checking the results of different tasks.

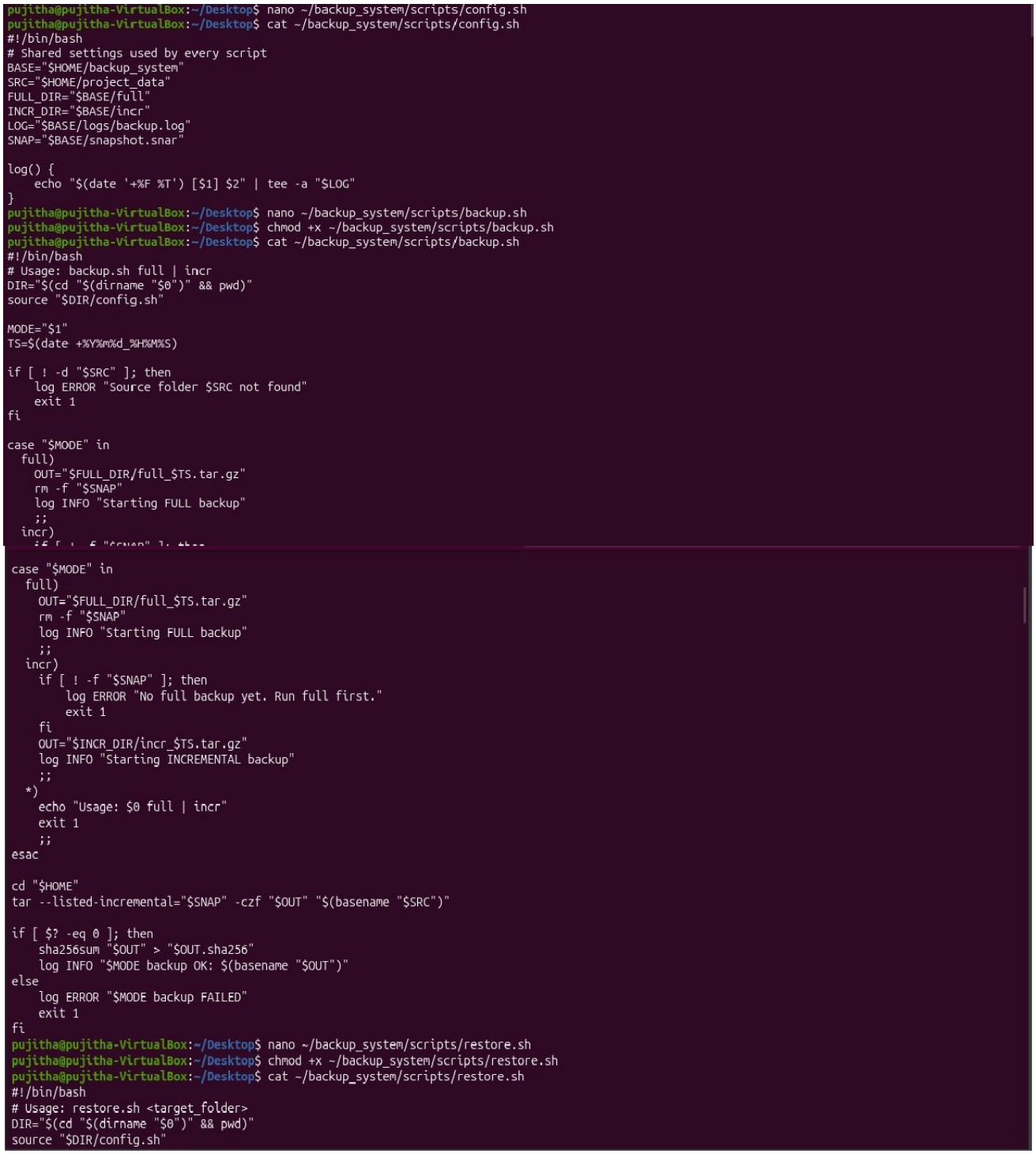
The project uses a menu-driven approach that allows users to select the required operation. Based on the selected option, the application performs the corresponding task and displays the result in the terminal. The main operations include full backup, incremental backup, listing available backups, scheduled backup management, viewing logs, restoring data, and testing recovery.

The Linux environment also supports task automation through the Cron scheduling service. Scheduled tasks help reduce the need for repeated manual execution. Logging provides a record of application activity, which can be reviewed when checking backup operations or investigating errors.

3.2.1 BACKEND PROCESSING

The backend is responsible for processing the user's selection and carrying out the corresponding backup or recovery operation. It connects the menu options with the Linux commands and shell-script operations required by the project. It also displays messages that help users understand the progress or result of an operation.

The backend plays an important role in coordinating backup creation, scheduled tasks, log management, and data restoration. Separating the processing logic from the user interface makes the project easier to understand and maintain.



```
pujitha@pujitha-VirtualBox:~/Desktop$ nano ~/backup_system/scripts/config.sh
pujitha@pujitha-VirtualBox:~/Desktop$ cat ~/backup_system/scripts/config.sh
#!/bin/bash
# Shared settings used by every script
BASE="$HOME/backup_system"
SRC="$HOME/project_data"
FULL_DIR="$BASE/full"
INCR_DIR="$BASE/incr"
LOG="$BASE/logs/backup.log"
SNAP="$BASE/snapshot.snar"

log() {
    echo "$(date '+%F %T') [$1] $2" | tee -a "$LOG"
}
pujitha@pujitha-VirtualBox:~/Desktop$ nano ~/backup_system/scripts/backup.sh
pujitha@pujitha-VirtualBox:~/Desktop$ chmod +x ~/backup_system/scripts/backup.sh
pujitha@pujitha-VirtualBox:~/Desktop$ cat ~/backup_system/scripts/backup.sh
#!/bin/bash
# Usage: backup.sh full | incr
DIR="$(cd "$(dirname "$0")" && pwd)"
source "$DIR/config.sh"

MODE="$1"
TS=$(date +%Y%m%d_%H%M%S)

if [ ! -d "$SRC" ]; then
    log ERROR "Source folder $SRC not found"
    exit 1
fi

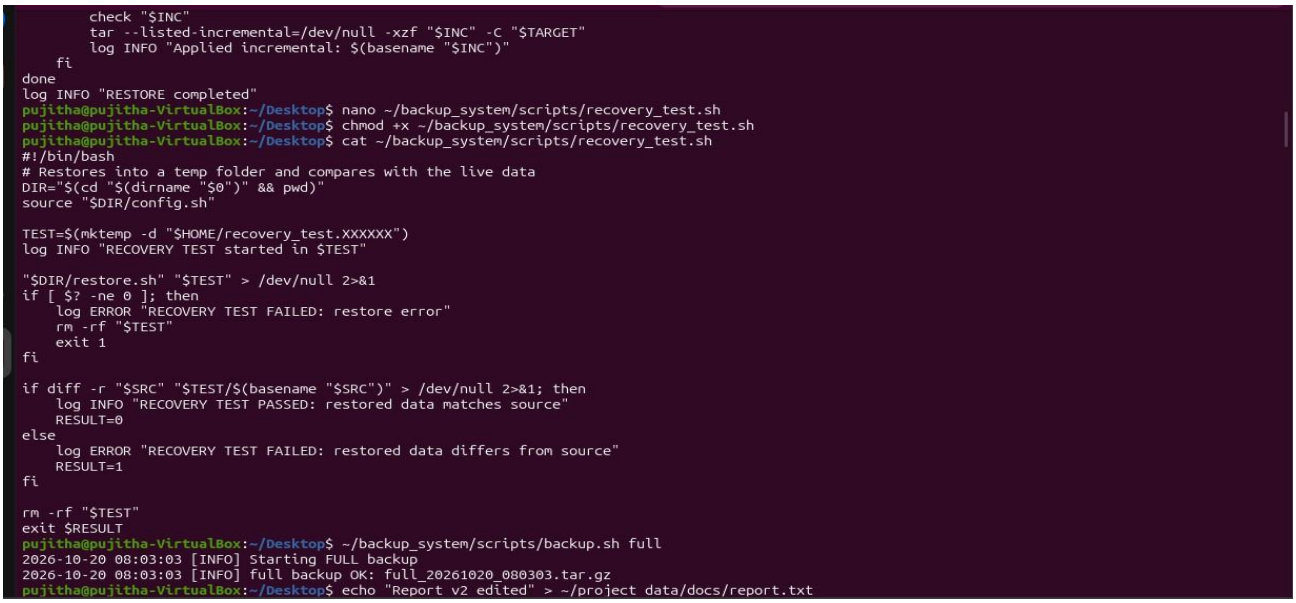
case "$MODE" in
  full)
    OUT="$FULL_DIR/full_$TS.tar.gz"
    rm -f "$SNAP"
    log INFO "Starting FULL backup"
    ;;
  incr)
    if [ ! -f "$SNAP" ]; then
        log ERROR "No full backup yet. Run full first."
        exit 1
    fi
    OUT="$INCR_DIR/incr_$TS.tar.gz"
    log INFO "Starting INCREMENTAL backup"
    ;;
  *)
    echo "Usage: $0 full | incr"
    exit 1
    ;;
esac

cd "$HOME"
tar --listed-incremental="$SNAP" -czf "$OUT" "$(basename "$SRC")"

if [ $? -eq 0 ]; then
    sha256sum "$OUT" > "$OUT.sha256"
    log INFO "$MODE backup OK: $(basename "$OUT")"
else
    log ERROR "$MODE backup FAILED"
    exit 1
fi
pujitha@pujitha-VirtualBox:~/Desktop$ nano ~/backup_system/scripts/restore.sh
pujitha@pujitha-VirtualBox:~/Desktop$ chmod +x ~/backup_system/scripts/restore.sh
pujitha@pujitha-VirtualBox:~/Desktop$ cat ~/backup_system/scripts/restore.sh
#!/bin/bash
# Usage: restore.sh <target_folder>
DIR="$(cd "$(dirname "$0")" && pwd)"
source "$DIR/config.sh"
```

Figure 3.2: Backend source code – configuration and setup

This screenshot shows the first part of the backend implementation. It provides evidence of the project scripts and the configuration logic used to prepare the backup and recovery operations. The configuration should be checked against the actual directory names and script names used when the project is executed.



```
        check "$INC"
        tar --listed-incremental=/dev/null -xzf "$INC" -C "$TARGET"
        log INFO "Applied incremental: $(basename "$INC")"
    fi
done
log INFO "RESTORE completed"
pujitha@pujitha-VirtualBox:~/Desktop$ nano ~/backup_system/scripts/recovery_test.sh
pujitha@pujitha-VirtualBox:~/Desktop$ chmod +x ~/backup_system/scripts/recovery_test.sh
pujitha@pujitha-VirtualBox:~/Desktop$ cat ~/backup_system/scripts/recovery_test.sh
#!/bin/bash
# Restores into a temp folder and compares with the live data
DIR="$(cd "$(dirname "$0")" && pwd)"
source "$DIR/config.sh"

TEST=$(mktemp -d "$HOME/recovery_test.XXXXXX")
log INFO "RECOVERY TEST started in $TEST"

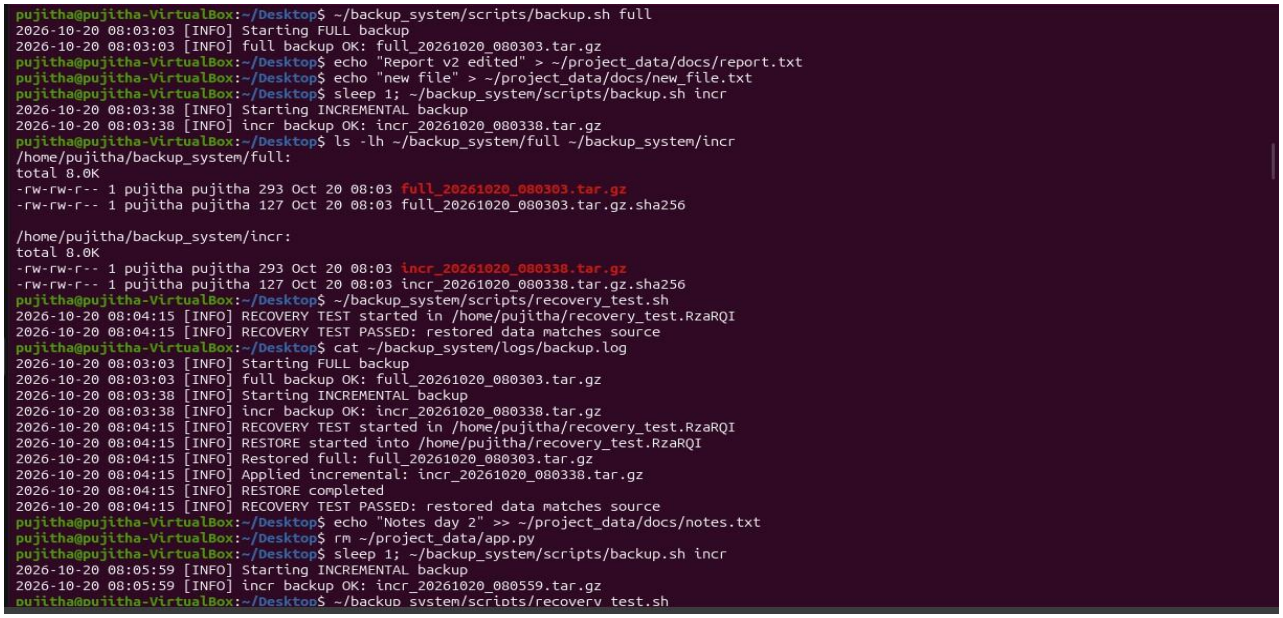
"$DIR/restore.sh" "$TEST" > /dev/null 2>&1
if [ $? -ne 0 ]; then
    log ERROR "RECOVERY TEST FAILED: restore error"
    rm -rf "$TEST"
    exit 1
fi

if diff -r "$SRC" "$TEST/$(basename "$SRC")" > /dev/null 2>&1; then
    log INFO "RECOVERY TEST PASSED: restored data matches source"
    RESULT=0
else
    log ERROR "RECOVERY TEST FAILED: restored data differs from source"
    RESULT=1
fi

rm -rf "$TEST"
exit $RESULT
pujitha@pujitha-VirtualBox:~/Desktop$ ~/backup_system/scripts/backup.sh full
2026-10-20 08:03:03 [INFO] Starting FULL backup
2026-10-20 08:03:03 [INFO] full backup OK: full_20261020_080303.tar.gz
pujitha@pujitha-VirtualBox:~/Desktop$ echo "Report v2 edited" > ~/project data/docs/report.txt
```

Figure 3.3: Backend source code – backup processing

This figure presents additional backend code used to process a backup-related task. It forms part of the implementation evidence and supports the explanation of how the application carries out the operation selected by the user.



```
pujitha@pujitha-VirtualBox:~/Desktop$ ~/backup_system/scripts/backup.sh full
2026-10-20 08:03:03 [INFO] Starting FULL backup
2026-10-20 08:03:03 [INFO] full backup OK: full_20261020_080303.tar.gz
pujitha@pujitha-VirtualBox:~/Desktop$ echo "Report v2 edited" > ~/project_data/docs/report.txt
pujitha@pujitha-VirtualBox:~/Desktop$ echo "new file" > ~/project_data/docs/new_file.txt
pujitha@pujitha-VirtualBox:~/Desktop$ sleep 1; ~/backup_system/scripts/backup.sh incr
2026-10-20 08:03:38 [INFO] Starting INCREMENTAL backup
2026-10-20 08:03:38 [INFO] incr backup OK: incr_20261020_080338.tar.gz
pujitha@pujitha-VirtualBox:~/Desktop$ ls -lh ~/backup_system/full ~/backup_system/incr
/home/pujitha/backup_system/full:
total 8.0K
-rw-rw-r-- 1 pujitha pujitha 293 Oct 20 08:03 full_20261020_080303.tar.gz
-rw-rw-r-- 1 pujitha pujitha 127 Oct 20 08:03 full_20261020_080303.tar.gz.sha256

/home/pujitha/backup_system/incr:
total 8.0K
-rw-rw-r-- 1 pujitha pujitha 293 Oct 20 08:03 incr_20261020_080338.tar.gz
-rw-rw-r-- 1 pujitha pujitha 127 Oct 20 08:03 incr_20261020_080338.tar.gz.sha256
pujitha@pujitha-VirtualBox:~/Desktop$ ~/backup_system/scripts/recovery_test.sh
2026-10-20 08:04:15 [INFO] RECOVERY TEST started in /home/pujitha/recovery_test.RzaRQI
2026-10-20 08:04:15 [INFO] RECOVERY TEST PASSED: restored data matches source
pujitha@pujitha-VirtualBox:~/Desktop$ cat ~/backup_system/logs/backup.log
2026-10-20 08:03:03 [INFO] Starting FULL backup
2026-10-20 08:03:03 [INFO] full backup OK: full_20261020_080303.tar.gz
2026-10-20 08:03:38 [INFO] Starting INCREMENTAL backup
2026-10-20 08:03:38 [INFO] incr backup OK: incr_20261020_080338.tar.gz
2026-10-20 08:04:15 [INFO] RECOVERY TEST started in /home/pujitha/recovery_test.RzaRQI
2026-10-20 08:04:15 [INFO] RESTORE started into /home/pujitha/recovery_test.RzaRQI
2026-10-20 08:04:15 [INFO] Restored full: full_20261020_080303.tar.gz
2026-10-20 08:04:15 [INFO] Applied incremental: incr_20261020_080338.tar.gz
2026-10-20 08:04:15 [INFO] RESTORE completed
2026-10-20 08:04:15 [INFO] RECOVERY TEST PASSED: restored data matches source
pujitha@pujitha-VirtualBox:~/Desktop$ echo "Notes day 2" >> ~/project_data/docs/notes.txt
pujitha@pujitha-VirtualBox:~/Desktop$ rm ~/project_data/app.py
pujitha@pujitha-VirtualBox:~/Desktop$ sleep 1; ~/backup_system/scripts/backup.sh incr
2026-10-20 08:05:59 [INFO] Starting INCREMENTAL backup
2026-10-20 08:05:59 [INFO] incr backup OK: incr_20261020_080559.tar.gz
pujitha@pujitha-VirtualBox:~/Desktop$ ~/backup_system/scripts/recovery_test.sh
```

Figure 3.4: Backend source code – file and backup handling

The output displays further backend commands and processing steps. These lines document how the project handles the relevant files or backup data. The screenshot should be read together with the backup execution output later in this chapter.

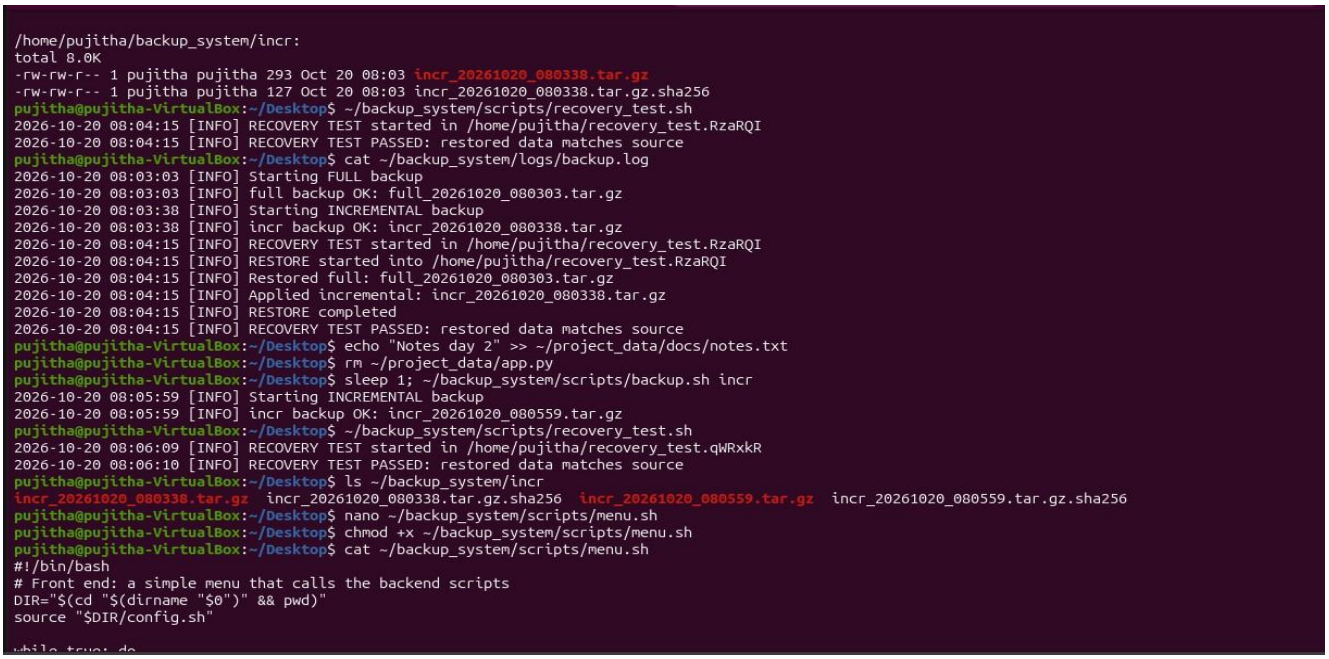


Figure 3.5: Backend source code – restoration and supporting functions

This screenshot shows another section of the backend implementation. It supports the report's description of the functions used by the system. The successful behavior of these functions is evaluated separately through the restoration and recovery-test outputs.

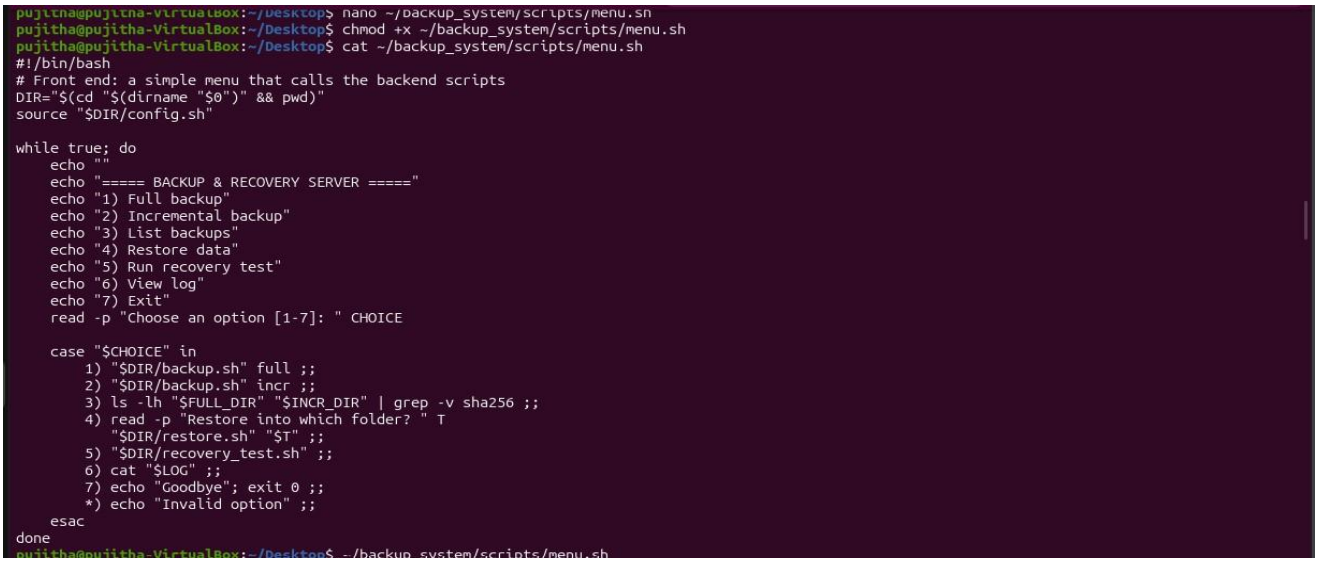


Figure 3.6: Backend source code – remaining processing logic

The figure records the remaining backend code included in the supplied screenshots. It provides a reference for understanding the program structure and the logic that connects the available operations.

3.2.2 FRONTEND AND MENU INTERFACE

The frontend of the Centralized Backup and Recovery Server is designed as a menu-driven terminal interface using Bash shell scripting. It allows users to interact with the backup system by selecting the required operation from the displayed menu. The interface makes the project easier to use because users can perform different backup and recovery operations without entering separate commands for every task.

The main menu provides options for performing a full backup, creating an incremental backup, listing available backups, managing scheduled backup tasks, viewing logs, restoring data, and testing the recovery process. Each option is connected to its corresponding backend operation.

When the user selects an option, the system processes the request and displays the result in the terminal. Successful operations provide confirmation messages, while invalid selections are handled through input validation. This helps users understand the status of each operation and reduces errors during system usage.

The menu-driven interface connects the user with the backend processing component. The frontend accepts user input, while the backend performs the requested backup, scheduling, logging, or recovery operation. This separation makes the system easier to operate, test, and maintain.

Frontend Implementation Output

The following screenshot demonstrates the menu-driven terminal interface of the Centralized Backup and Recovery Server.

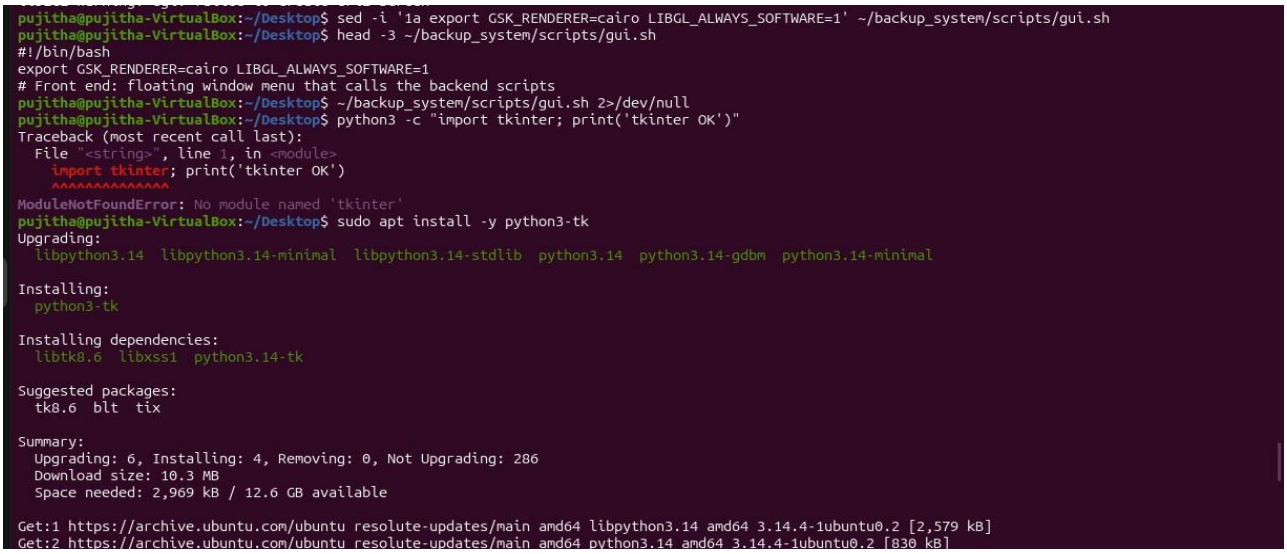


Figure 3.7: Frontend source code – interface layout

This screenshot shows the first part of the frontend implementation. It documents how the interface is arranged and how its main components are created. The code provides the structure through which a user can access the project's functions.

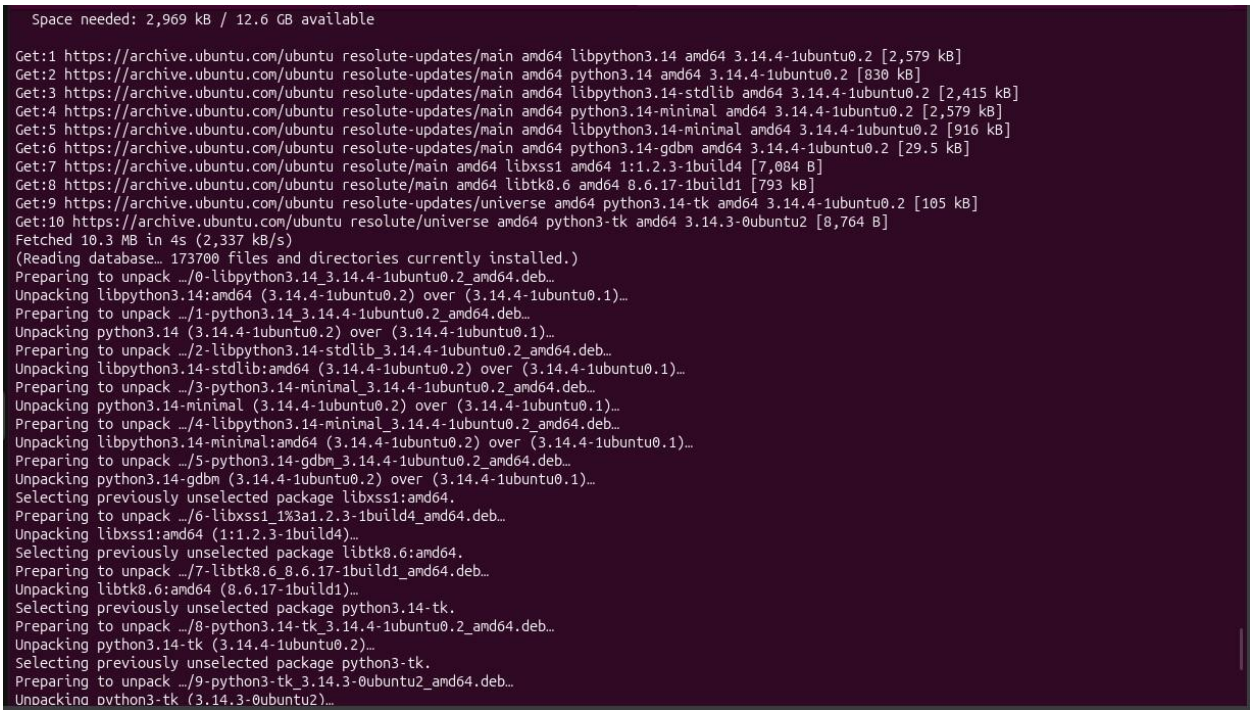


Figure 3.8: Frontend source code – controls and display area

The figure shows further frontend code associated with the interface controls and display area. It supports the explanation of how operation choices and output messages are presented to the user.

```
Unpacking libpython3.14-minimal:amd64 (3.14.4-1ubuntu0.2) over (3.14.4-1ubuntu0.1)…
Preparing to unpack …/5-python3.14-gdbm_3.14.4-1ubuntu0.2_amd64.deb…
Unpacking python3.14-gdbm (3.14.4-1ubuntu0.2) over (3.14.4-1ubuntu0.1)…
Selecting previously unselected package libxss1:amd64.
Preparing to unpack …/6-libxss1_1%3a1.2.3-1build4_amd64.deb…
Unpacking libxss1:amd64 (1:1.2.3-1build4)…
Selecting previously unselected package libtk8.6:amd64.
Preparing to unpack …/7-libtk8.6_8.6.17-1build1_amd64.deb…
Unpacking libtk8.6:amd64 (8.6.17-1build1)…
Selecting previously unselected package python3.14-tk.
Preparing to unpack …/8-python3.14-tk_3.14.4-1ubuntu0.2_amd64.deb…
Unpacking python3.14-tk (3.14.4-1ubuntu0.2)…
Selecting previously unselected package python3-tk.
Preparing to unpack …/9-python3-tk_3.14.3-0ubuntu2_amd64.deb…
Unpacking python3-tk (3.14.3-0ubuntu2)…
Setting up libpython3.14-minimal:amd64 (3.14.4-1ubuntu0.2)…
Setting up python3.14-gdbm (3.14.4-1ubuntu0.2)…
Setting up libxss1:amd64 (1:1.2.3-1build4)…
Setting up python3.14-minimal (3.14.4-1ubuntu0.2)…
Setting up libpython3.14-stdlib:amd64 (3.14.4-1ubuntu0.2)…
Setting up libtk8.6:amd64 (8.6.17-1build1)…
Setting up libpython3.14:amd64 (3.14.4-1ubuntu0.2)…
Setting up python3.14 (3.14.4-1ubuntu0.2)…
Setting up python3.14-tk (3.14.4-1ubuntu0.2)…
Setting up python3-tk (3.14.3-0ubuntu2)…
Processing triggers for man-db (2.13.1-1build1)…
Processing triggers for desktop-file-utils (0.28-1build1)…
Processing triggers for gnome-menus (3.38.1-1ubuntu1)…
Processing triggers for libc-bin (2.43-2ubuntu2.3)…
Processing triggers for systemd (259.5-0ubuntu3.4)…
pujitha@pujitha-VirtualBox:~/Desktop$ python3 -c "import tkinter; print('tkinter OK')"
tkinter OK
pujitha@pujitha-VirtualBox:~/Desktop$ nano ~/backup_system/scripts/dashboard.py
pujitha@pujitha-VirtualBox:~/Desktop$ chmod +x ~/backup_system/scripts/dashboard.py
pujitha@pujitha-VirtualBox:~/Desktop$ python3 ~/backup_system/scripts/dashboard.py
pujitha@pujitha-VirtualBox:~/Desktop$ python3 ~/backup_system/scripts/dashboard.py
pujitha@pujitha-VirtualBox:~/Desktop$
```

Figure 3.9: Frontend source code – operation controls

This screenshot records code for additional interface controls. These controls provide access to project operations. The exact action performed by each control should correspond to the implementation and to the relevant result screenshot.

```
pujitha@pujitha-VirtualBox:~/Desktop$ crontab -e
crontab: installing new crontab
pujitha@pujitha-VirtualBox:~/Desktop$ crontab -l

#
# Each task to run has to be defined through a single line
# indicating with different fields when the task will be run
# and what command to run for the task
#
# To define the time you can provide concrete values for
# minute (m), hour (h), day of month (dom), month (mon),
# and day of week (dow) or use '*' in these fields (for 'any').
#
# Notice that tasks will be started based on the cron's system
# daemon's notion of time and timezones.
#
# Output of the crontab jobs (including errors) is sent through
# email to the user the crontab file belongs to (unless redirected).
#
# For example, you can run a backup of all your user accounts
# at 5 a.m every week with:
# 0 5 * * 1 tar -zcf /var/backups/home.tgz /home/
#
# For more information see the manual pages of crontab(5) and cron(8)
#
# m h  dom mon dow   command
0 2 * * 0 /home/pujitha/backup_system/scripts/backup.sh full >> /home/pujitha/backup_system/logs/cron.log 2>&1
0 2 * * 1-6 /home/pujitha/backup_system/scripts/backup.sh incr >> /home/pujitha/backup_system/logs/cron.log 2>&1
pujitha@pujitha-VirtualBox:~/Desktop$ systemctl is-active cron
active
```

Figure 3.10: Frontend source code – event handling

The output documents another part of the frontend code used to handle user interactions. Event handling connects a user's selection with the relevant operation and allows the result to be shown in the interface.

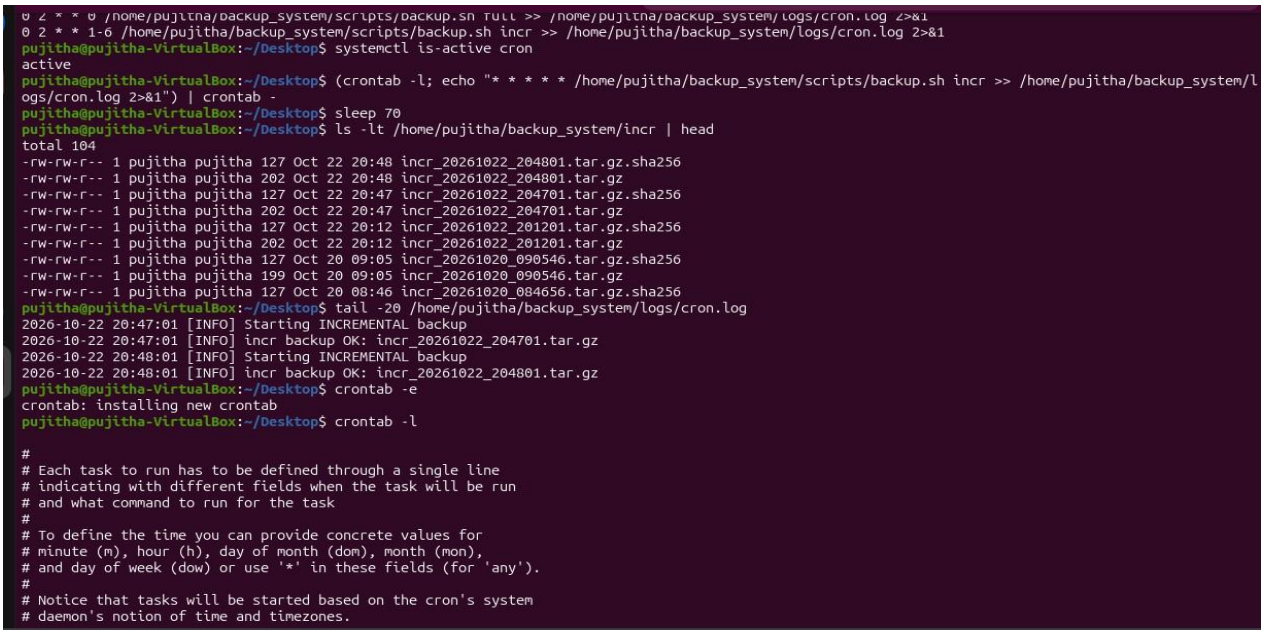


Figure 3.11: Frontend source code – output handling

This figure shows code associated with displaying operation output or messages. Clear output messages help users understand whether a requested action has been started or completed.

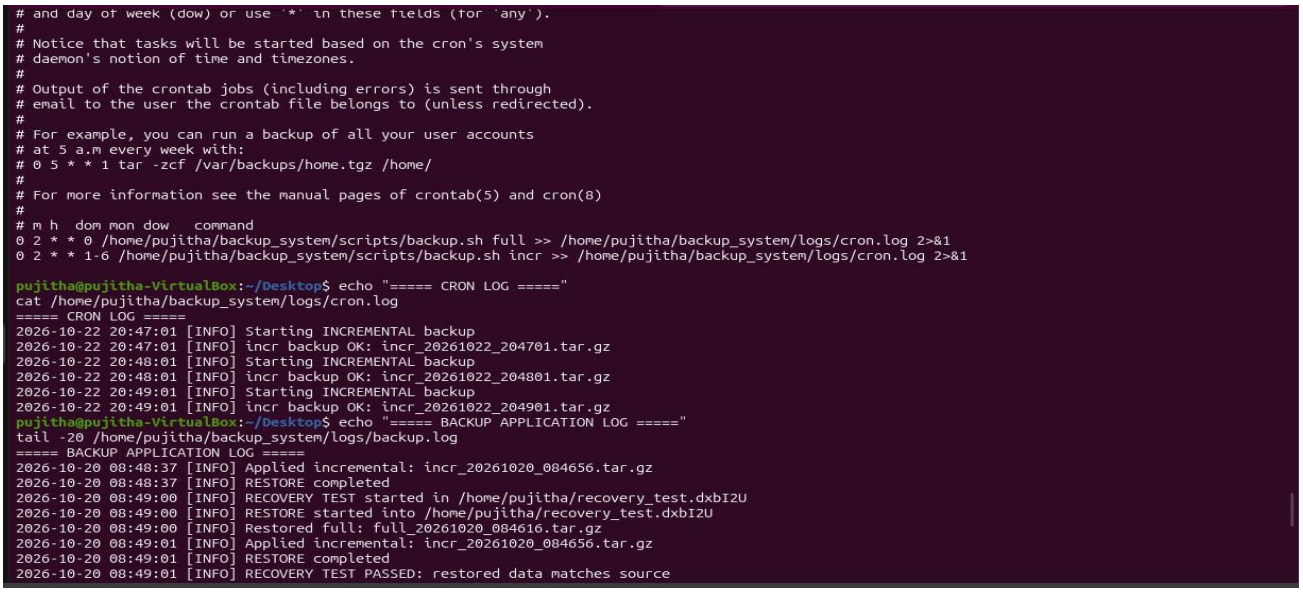


Figure 3.12: Frontend source code – backup-related controls

The screenshot documents interface code related to backup functions. It complements the backend code and shows the user-facing side of the application.

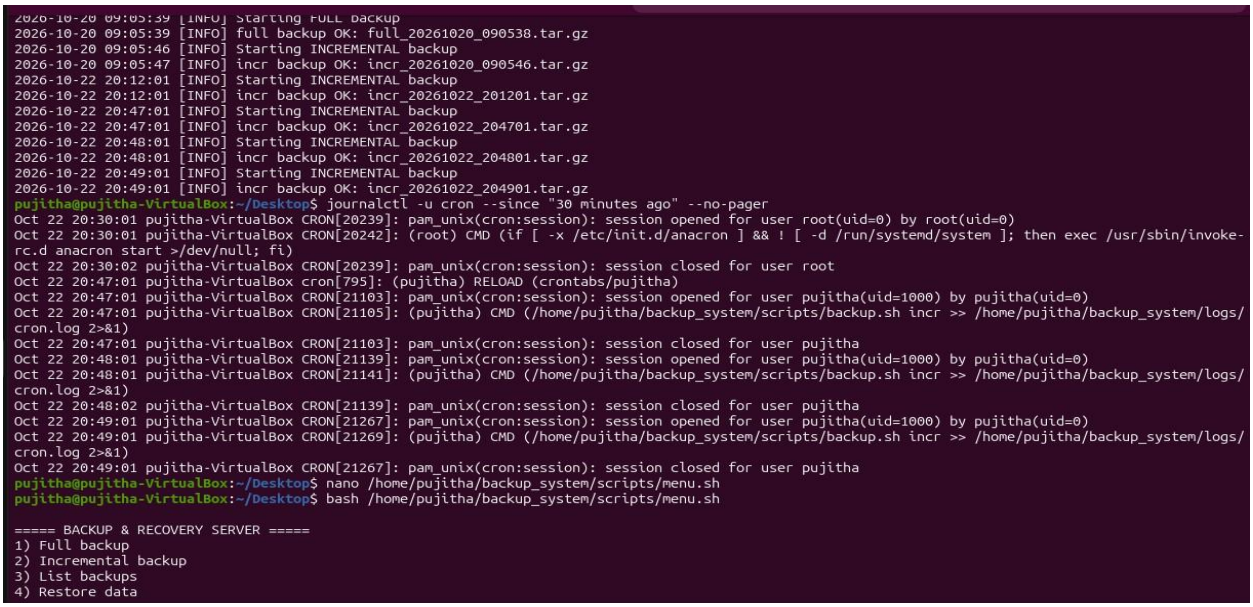


Figure 3.13: Frontend source code – interface components

This figure presents additional interface implementation. It is included as code evidence and should be read together with the dashboard screenshot and the backup-operation outputs.

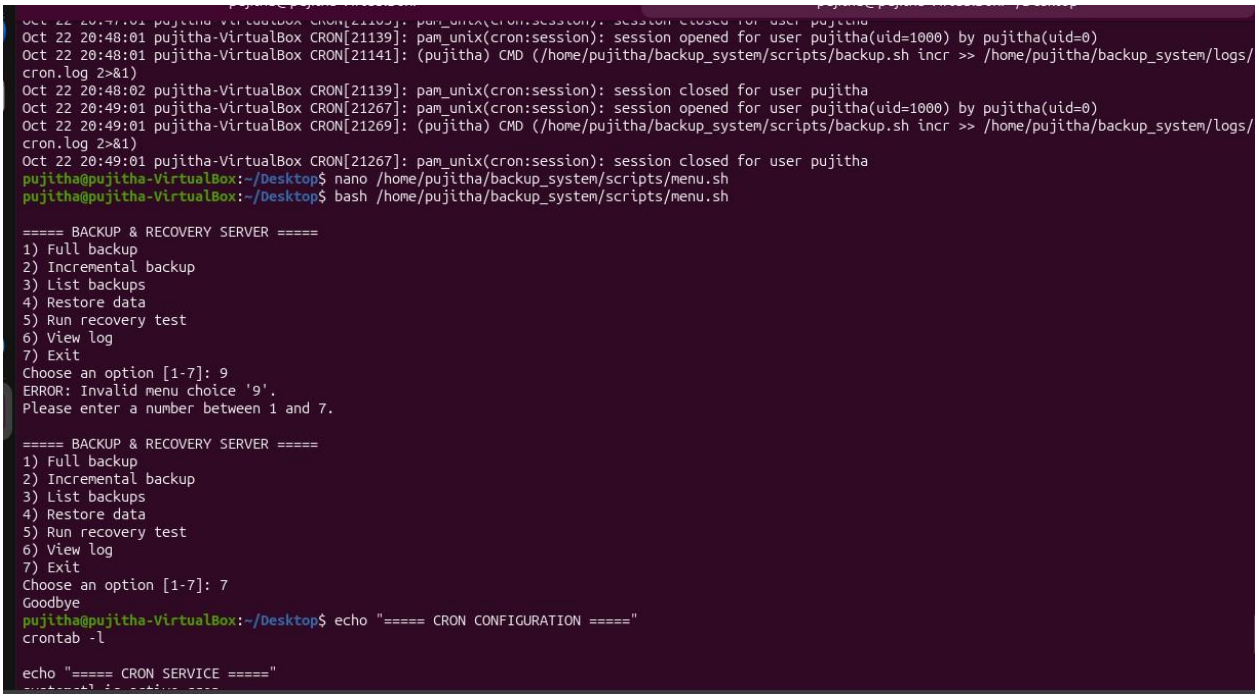


Figure 3.14: Frontend source code – operation wiring

The screenshot records further code that connects interface elements to the application's functions. This connection allows the user to request an operation without manually entering every supporting command.

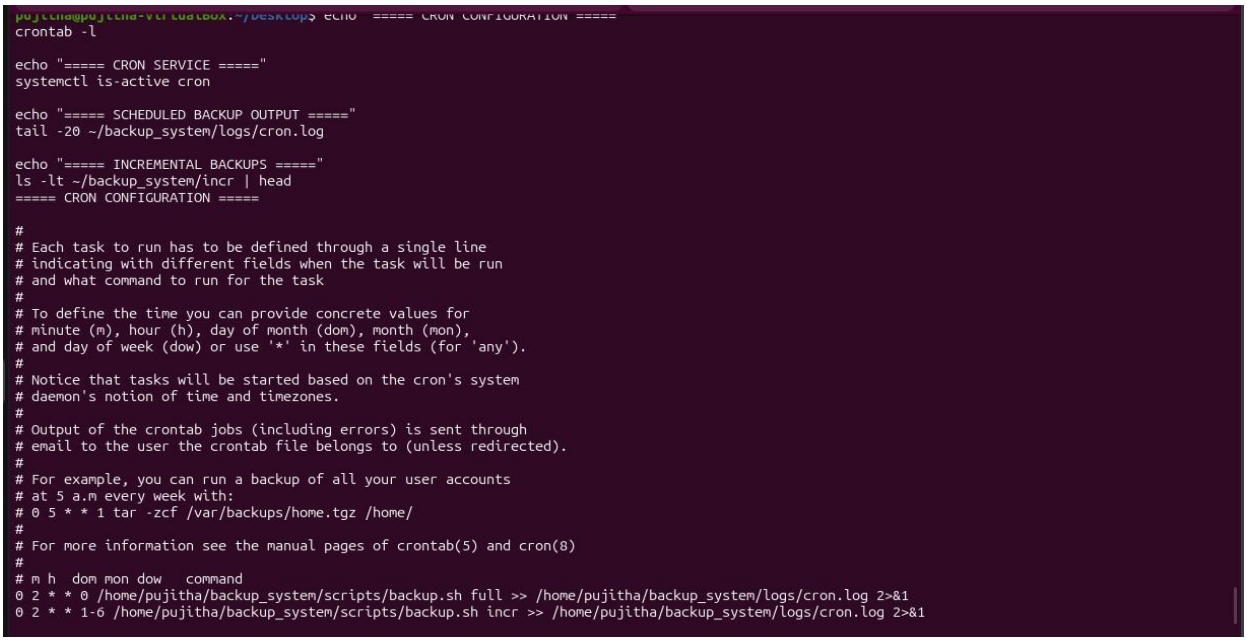


Figure 3.15: Frontend source code – messages and interaction

This figure documents more of the interface interaction logic. It helps explain how the application communicates with the user when different operations are selected.

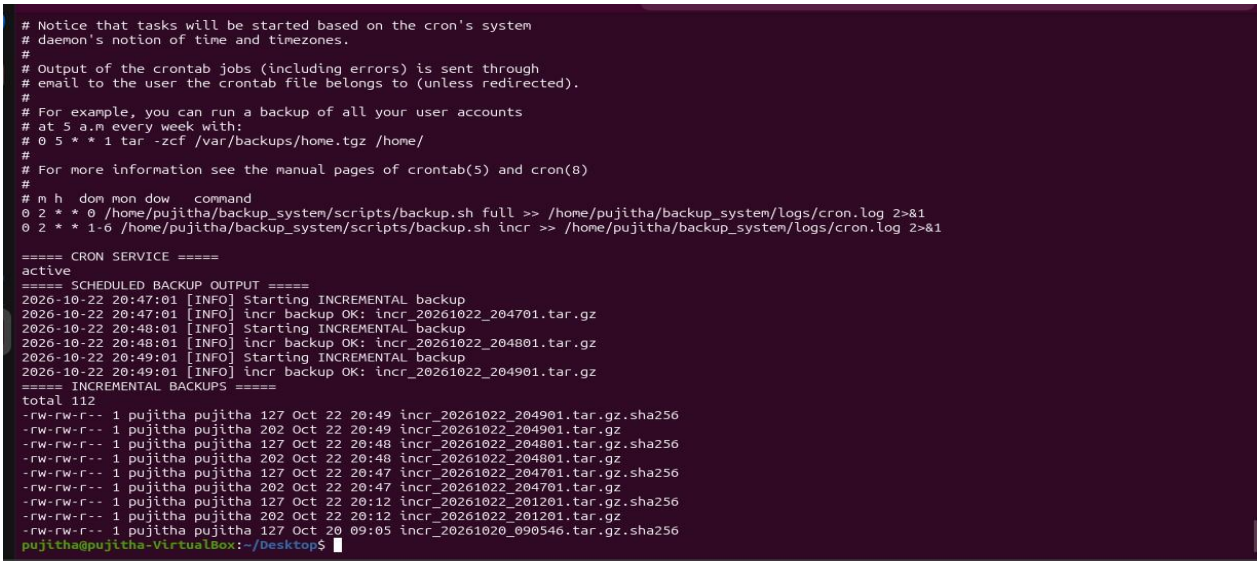


Figure 3.16: Frontend source code – completed interface implementation

The screenshot shows the final part of the supplied frontend code. Together, Figures 3.7–3.16 document the implementation used to present the backup and recovery options. It provides visual evidence of the interface developed for the project.

3.2.3 FULL BACKUP IMPLEMENTATION

A full backup is a process of creating a complete copy of the selected data at a particular time. It provides a separate copy of the original files that can be used when the original data is accidentally deleted, damaged, or lost. In the Centralized Backup and Recovery Server, the full backup operation is implemented using Linux shell scripting and backup utilities.

When the user selects the Full Backup option from the menu, the system starts the backup process for the configured source data. The selected files and directories are copied into a backup archive. The archive is stored in the configured backup location so that it can be accessed later during restoration.

The full backup operation helps preserve the complete set of selected data. It also provides a baseline for backup management and recovery operations. The system displays the operation status after processing the request, allowing the user to check whether the backup was completed successfully.

The backup result can be verified by checking the generated backup archive and confirming that the expected files are included. Maintaining backup files in an organized location makes it easier to identify and select the required backup during data recovery.

Full Backup Implementation

A full backup is intended to create a complete backup copy of the configured source data. After starting the operation, the user should review the output message and confirm that the backup file appears in the configured backup location. The screenshot below is the supplied evidence for the full-backup operation.

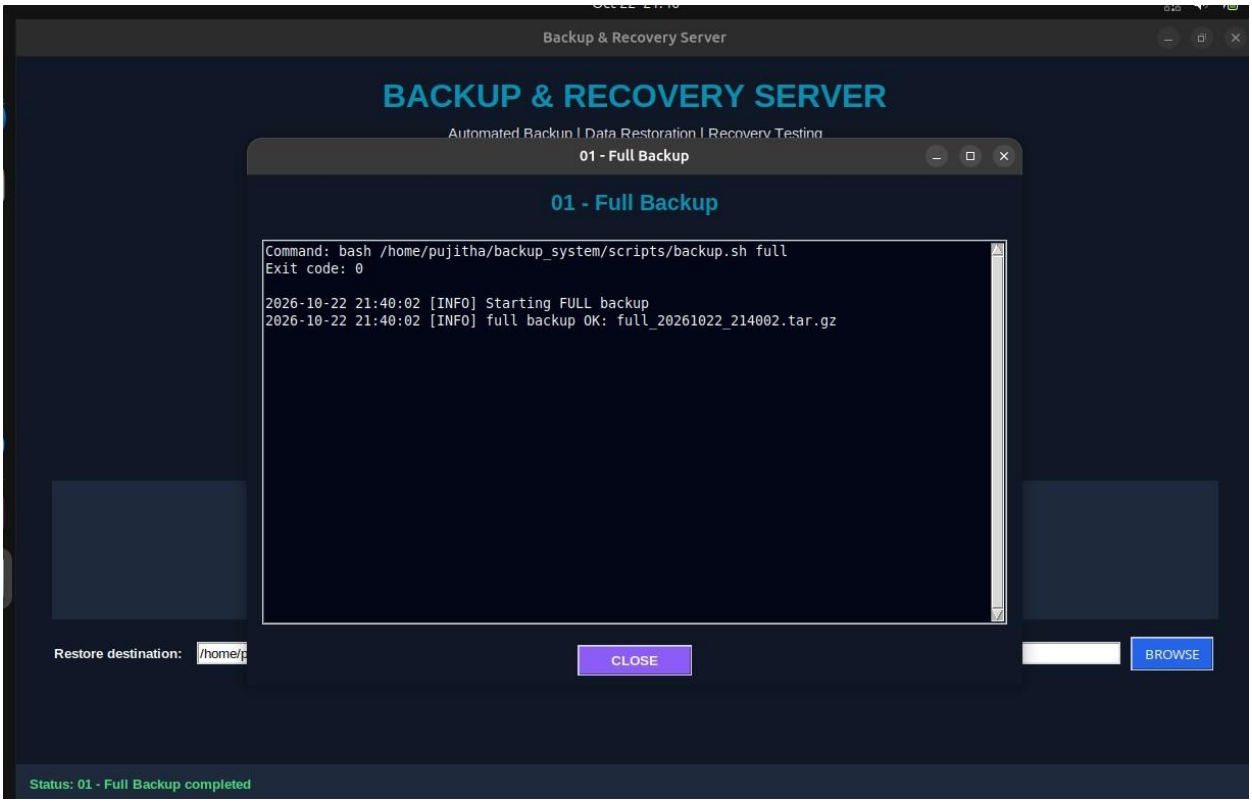


Figure 3.17: Full Backup Operation Output

The screenshot shows the dashboard during the full-backup operation. The output area provides the visible result of the selected task. This figure should be used to explain the message displayed by the program and, where visible, the backup file or operation status. A backup should be reported as successful only if the displayed output confirms completion.

Result

The full backup functionality provides a complete backup copy of the selected data. This backup can be used as a recovery source when the original data needs to be restored.

3.2.4 INCREMENTAL BACKUP IMPLEMENTATION

An incremental backup is a backup method that saves data changed or added since the previous backup. Unlike a full backup, which copies all selected data each time, an incremental backup aims to reduce the amount of data copied during subsequent backup operations. This can help save storage space and reduce backup time.

In the Centralized Backup and Recovery Server, the incremental backup option allows the user to initiate the incremental backup process through the menu-driven terminal interface. The system processes the configured source data and stores the resulting backup in the designated backup location.

Incremental backups are useful when data changes frequently and regular backups are required. By saving changes rather than repeatedly copying all selected data, the system can make backup operations more efficient. However, successful recovery may depend on having the required earlier backup and subsequent incremental backups available.

After the operation, the user can check the terminal output to review the status of the backup process. The generated backup should also be checked to confirm that it exists in the expected location.

Incremental Backup Implementation

An incremental backup is intended to store changes according to the method implemented in the project. It can reduce repeated copying when compared with creating a complete copy every time. The generated output should be inspected to verify that the operation was carried out and that the resulting backup is available.

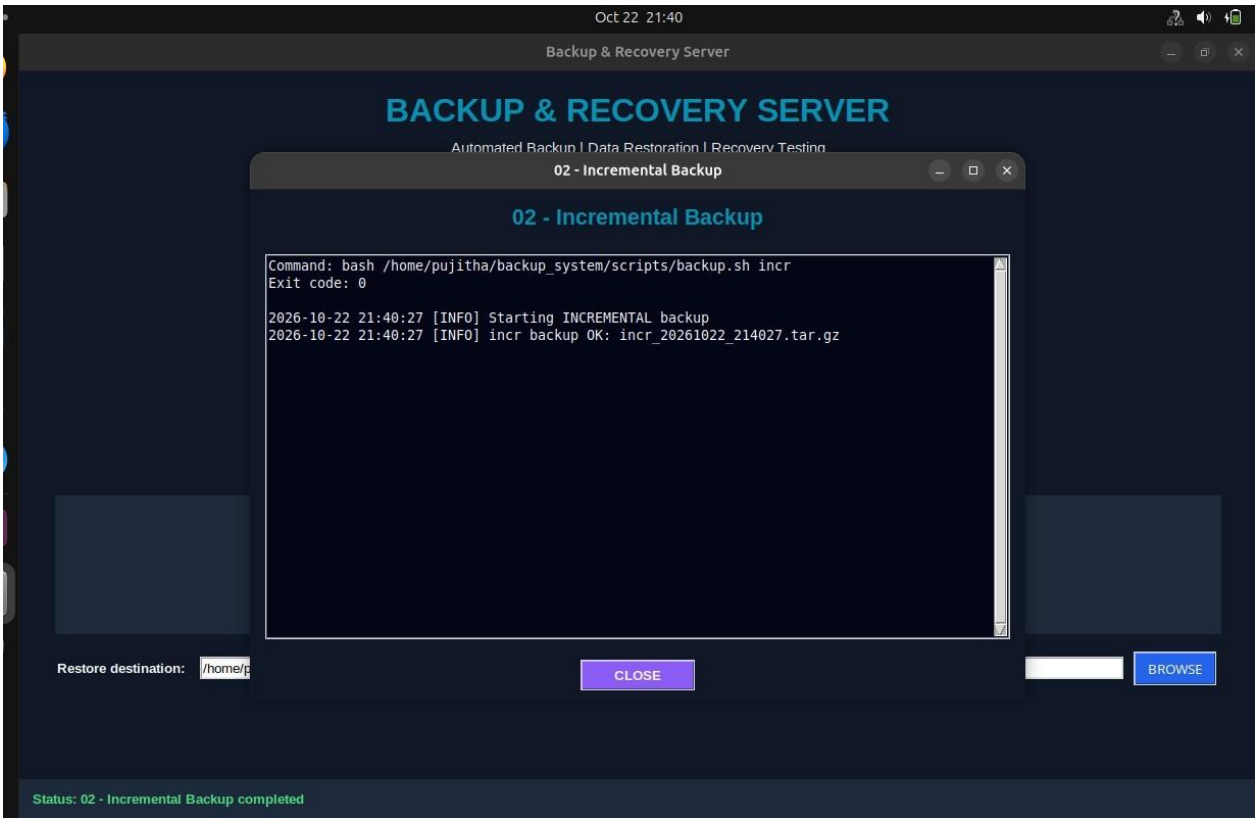


Figure 3.18: Incremental Backup Operation Output

This screenshot documents the incremental-backup option in the project interface. It provides evidence of the operation selected and the result shown by the application. The explanation of the incremental method must follow the actual script logic rather than assuming a particular comparison or snapshot technique.

Result

The incremental backup functionality provides a method for backing up changes to the selected data. When used with the required earlier backups, it supports efficient storage management and data recovery.

3.2.5 LISTING AND MANAGING BACKUPS

Listing and managing backups is an important function of the Centralized Backup and Recovery Server. As backups are created over time, users need a convenient way to identify the available backup files and select the appropriate backup for recovery.

The system provides a List Backups option through its menu-driven terminal interface. When the user selects this option, the system displays the available backup files stored in the configured backup location. This helps the user check whether backup files have been created and are available for further operations.

Backup listing also supports backup management by making it easier to identify the available archives. Users can review the displayed backup names and locate the required backup before starting a restoration operation. Keeping backup files organized helps simplify administration and reduces confusion when multiple backups are available.

The backup listing function complements the full and incremental backup features. After creating a backup, the user can return to the menu and list the available backups to verify that the backup file is present. The displayed information depends on the backup files available in the configured location.

Listing Backups – Implementation Output

The backup-listing option helps users inspect backup files that have already been created. Listing available backups makes it easier to select a backup for later restoration and to check whether expected files are present.

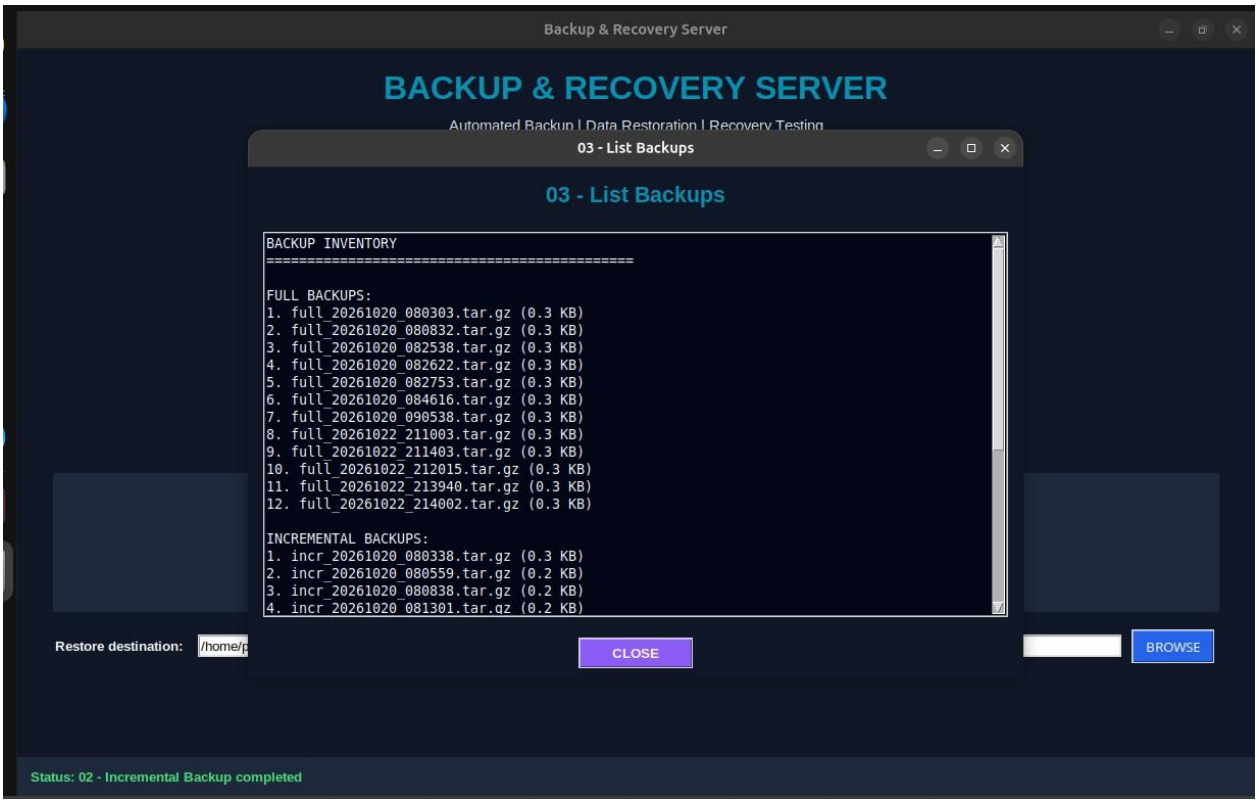


Figure 3.19: Listing Available Backup Files

The screenshot shows the list-backups function and the output area containing the available backup records or filenames. It helps demonstrate that users can inspect the stored backup items through the interface. The visible filenames and details should be used as the evidence when describing the result.

Result

The List Backups function helps users identify available backup files and locate the required archive for subsequent recovery operations. It improves the organization and accessibility of backup data.

3.2.6 SCHEDULED BACKUP MANAGEMENT USING CRON

Scheduled backup management is an important feature of the Centralized Backup and Recovery Server.

It allows backup operations to be arranged to run automatically at specified times instead of requiring the user to start every backup manually.

Linux provides a scheduling utility called Cron, which executes commands or scripts according to a configured schedule. A scheduled task is defined using a Cron job. The schedule specifies when the selected script should run, such as at a particular time or at regular intervals.

In this project, the scheduled backup feature allows the user to access backup scheduling options through the menu-driven terminal interface. This makes it easier to manage scheduled backup operations and helps maintain a regular backup routine.

Automatic scheduling can reduce repetitive manual work and improve the consistency of backup operations. However, the scheduled task must be configured correctly, and the system must be available when the task is due to run. The execution results can be checked through the project's logs.

Cron Job Menu – Implementation Output

Cron is a Linux scheduling service that can run configured commands or scripts at specified times. The project interface includes a scheduling-related menu. The existence of a menu option alone does not prove that a schedule is active; the configured job and its execution logs should also be checked.

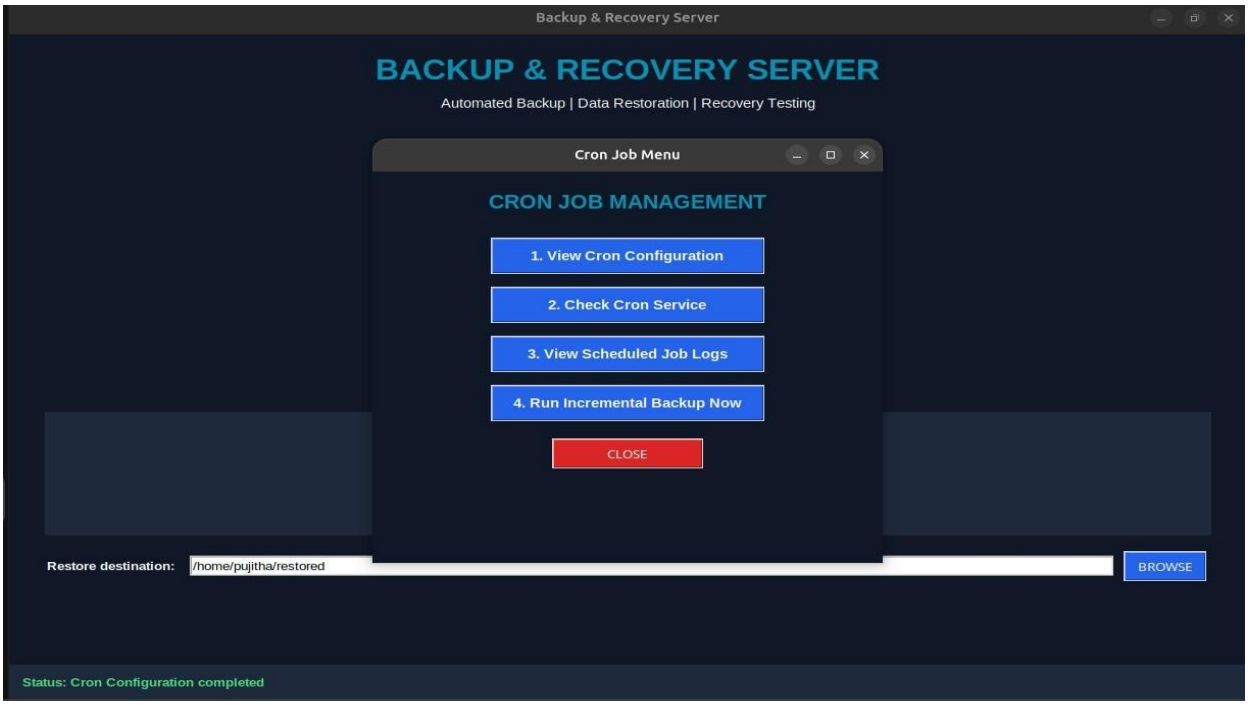


Figure 3.20: Cron Job Menu for Scheduled Backup Management

This figure shows the scheduling-related menu available in the application. It documents how the user accesses Cron or scheduled-backup functions. The actual schedule, if enabled, must be stated according to the Cron configuration used in the project.

Result

The Cron Job Menu provides an interface for managing scheduled backup operations. It supports automation and reduces the need to initiate every backup manually.

3.2.7 EXECUTING BACKUP OPERATIONS

The backup execution function allows users to initiate a backup operation through the Centralized Backup and Recovery Server. It provides a convenient way to run the configured backup process from the menu-driven terminal interface.

When the user selects the Run Backup option, the system processes the request and executes the corresponding backup operation. The script performs the configured tasks and displays the result in the terminal. This helps the user determine whether the operation completed successfully or whether further checking is required.

Executing backup operations is an important part of data protection because regular backups help preserve important files and directories. The user can initiate a backup when required, while scheduled backup tasks can support automatic execution at configured intervals.

After execution, the generated backup should be checked to confirm that the expected backup file is available. The application logs can also be reviewed to examine recorded events and help identify any errors that occurred during processing.

Run Backup – Implementation Output

Manual execution allows a user to initiate a backup when required rather than waiting for a scheduled run. The output can be reviewed to determine whether the requested operation completed and to identify any displayed errors.

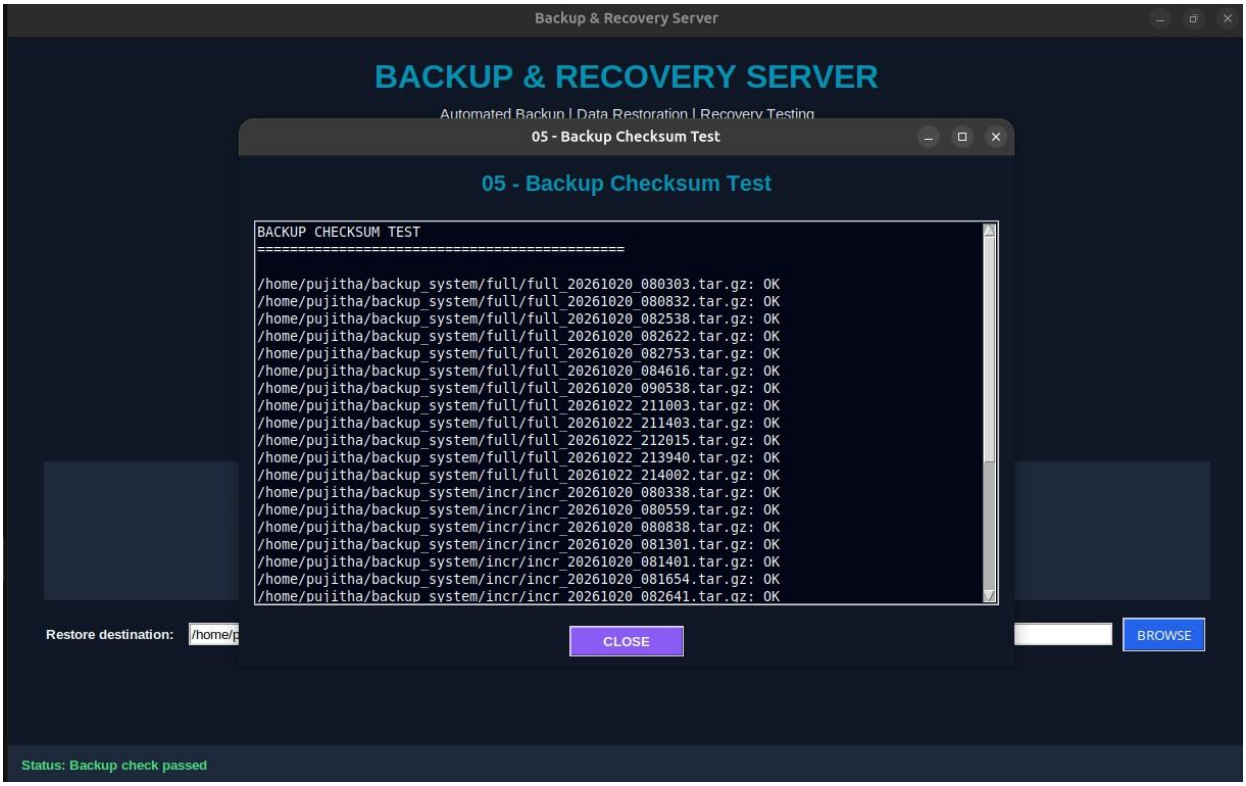


Figure 3.21: Execution of the Backup Operation

The screenshot shows the application output associated with running a backup. It provides evidence of the manual execution workflow and the result reported by the interface. The report should quote or describe only the status that is visible in the screenshot.

Result

The Run Backup function provides a convenient way to initiate backup processing. Together with backup listing, scheduling, and logging, it supports the overall backup management process.

3.2.8 VIEWING SCHEDULED BACKUP LOGS

Scheduled backup logs help users check the activities performed by the backup system. They provide useful information about scheduled operations and help users determine whether backup tasks have been executed as expected.

In the Centralized Backup and Recovery Server, the Scheduled Logs option allows users to view the available records related to scheduled backup activities. These records help track the execution of backup tasks and provide information for reviewing the system's operation.

Log monitoring is important because a scheduled task may not always complete as expected. Reviewing the available logs can help users identify recorded errors, check the execution status, and investigate problems that may affect the backup process.

The scheduled logging feature complements the Cron scheduling mechanism. Cron is responsible for running configured tasks according to their schedules, while the logs provide a way to review the recorded activity. Together, these features help improve the monitoring and management of scheduled backups.

Scheduled Logs – Implementation Output

Scheduled logs provide a record of activity associated with scheduled jobs. They can help a user determine whether scheduled operations have run and whether any messages or errors were recorded.

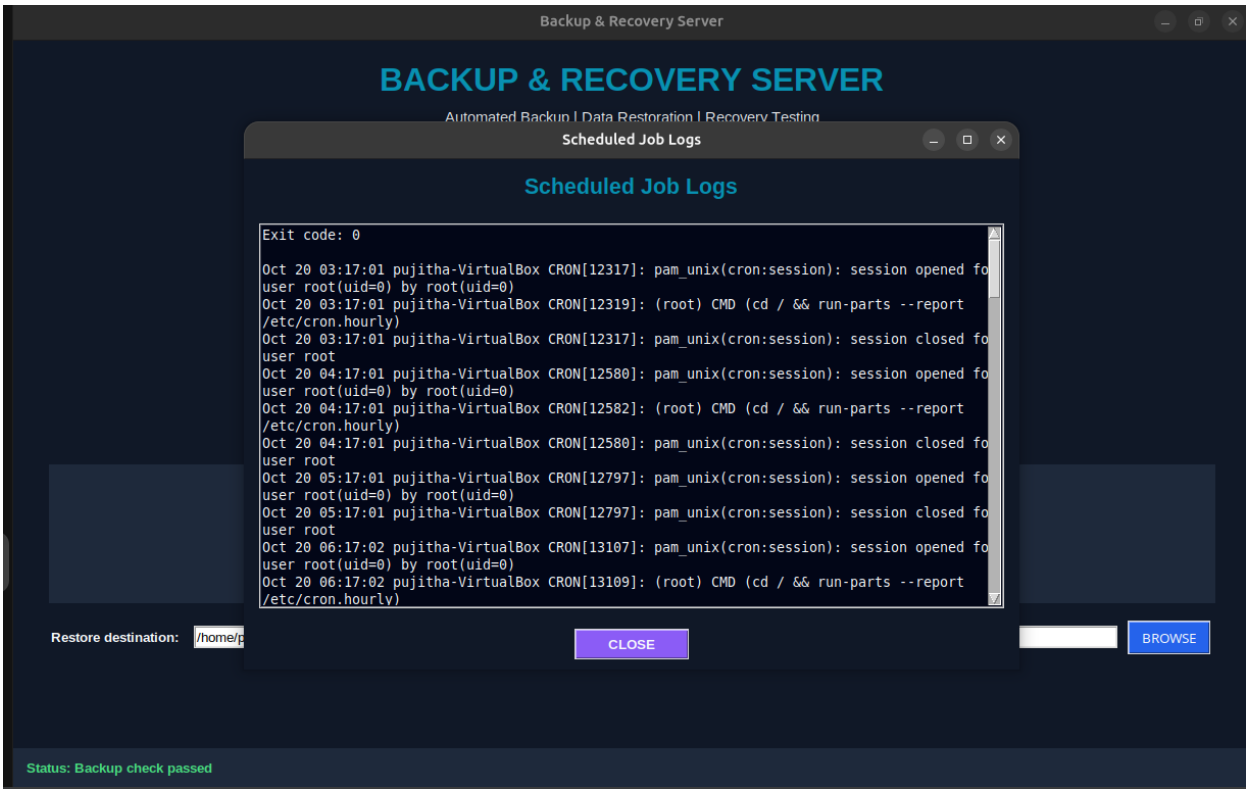


Figure 3.22: Scheduled Backup Logs Display

This figure displays the scheduled-log view in the application. The output can be used to inspect the records shown by the system. If the log includes timestamps or job status messages, these can be discussed as evidence of the recorded activity.

Result

The Scheduled Logs function helps users review recorded scheduled backup activities. It supports monitoring, troubleshooting, and verification of the backup scheduling process.

3.2.9 APPLICATION LOG MANAGEMENT

Application logs record information generated while the program is running. In the Centralized Backup and Recovery Server, the main operations – starting a backup, completing a backup, starting a restoration and running a recovery test – leave a time-stamped entry in the application log. Reviewing this log helps users understand which operations were carried out in the past and investigate problems without relying only on the information that is currently visible on the screen.

The dashboard provides a dedicated option for viewing the application log (option 07). When it is selected, the log opens in a separate window titled “07 - Application Log”. The records are shown in a scrollable text area with a Close button below it. Each record begins with a date and time and a severity label such as [INFO], followed by a short message that describes the event.

Application logging complements the scheduled logging described in Section 3.2.8. The scheduled job logs show the records produced by the Cron service, whereas the application log shows the records written by the backup, restore and recovery-test operations themselves. Used together, the two views give a more complete picture of what the system has done.

Application Log – Implementation Output

The following screenshot shows the application log as displayed by the Application Log option of the dashboard.

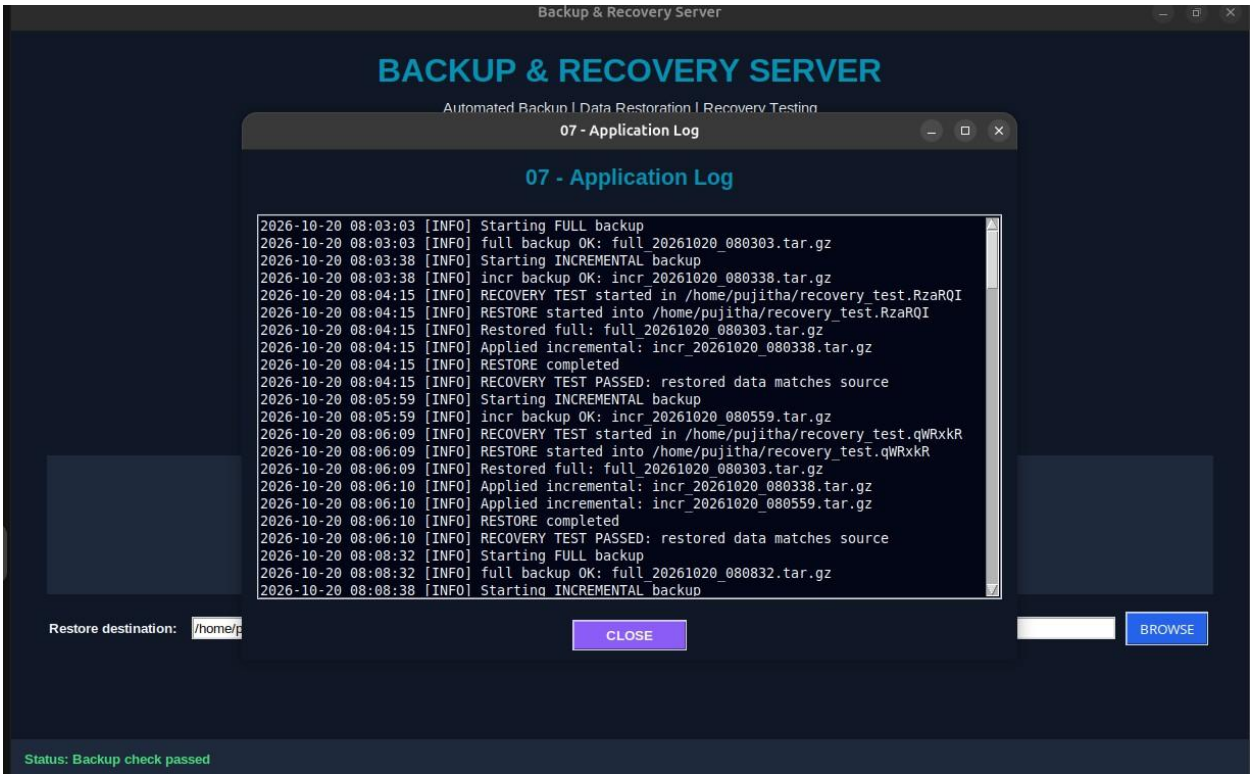


Figure 3.23: Application Log Output

The screenshot shows the application log window with entries dated 2026-10-20. The first entries record a full backup (full_20261020_080303.tar.gz) and an incremental backup (incr_20261020_080338.tar.gz), both reported as OK. They are followed by a recovery test that was started in a temporary directory under /home/pujitha. Within this test the log records the start of a restore, the restoration of the full backup, the application of the incremental backup, the completion of the restore, and the message “RECOVERY TEST PASSED: restored data matches source”.

The same pattern appears again further down the log. After another incremental backup (incr_20261020_080559.tar.gz), a second recovery test restores the full backup and applies both incremental archives before reporting that the test passed. The final visible entries record the start of another full backup and the start of another incremental backup at 08:08:32 and 08:08:38. The status bar of the dashboard displays “Status: Backup check passed”.

Result

The Application Log function allows the user to review backup, restore and recovery-test activity with timestamps. The entries also show which full and incremental archives were used during a restore, which makes the log useful for troubleshooting and for verifying the order in which operations were performed.

3.2.10 DATA RESTORATION

Restoration is the process of recovering data from an available backup and placing it in a selected destination. It is the purpose for which backups are created, so the restoration procedure must be reliable and must not damage existing data by accident. The destination should therefore be chosen carefully so that restored files do not unintentionally overwrite the original source data.

In the dashboard, the destination is entered in the “Restore destination” field at the bottom of the window. The field can be edited directly or filled by using the Browse button. In the screenshots included in this report, the destination is /home/pujitha/restored. The restore operation is selected as option 08 of the main menu, and its result is displayed in a window titled “08 - Restore Result”.

Before the restoration is carried out, the application displays prompts that ask the user to confirm the action. Two such prompts are captured in the screenshots. The “Confirm Restore” prompt states the destination folder and asks whether to continue. The “Destination Not Empty” prompt warns that the destination already contains files and that existing files may be overwritten. Both prompts offer Yes and No buttons, so the user can cancel the operation before any data is changed.

The restore result shows that restoration is performed in two stages. The full backup is restored first, and the incremental backup is then applied on top of it. This follows the relationship between full and incremental backups described in Sections 3.2.3 and 3.2.4: the full backup provides the baseline, and the incremental backup adds the changes recorded after it.

Data Restoration – Implementation Output

The following screenshots show the prompts, the completion message and the result window of the restoration workflow.

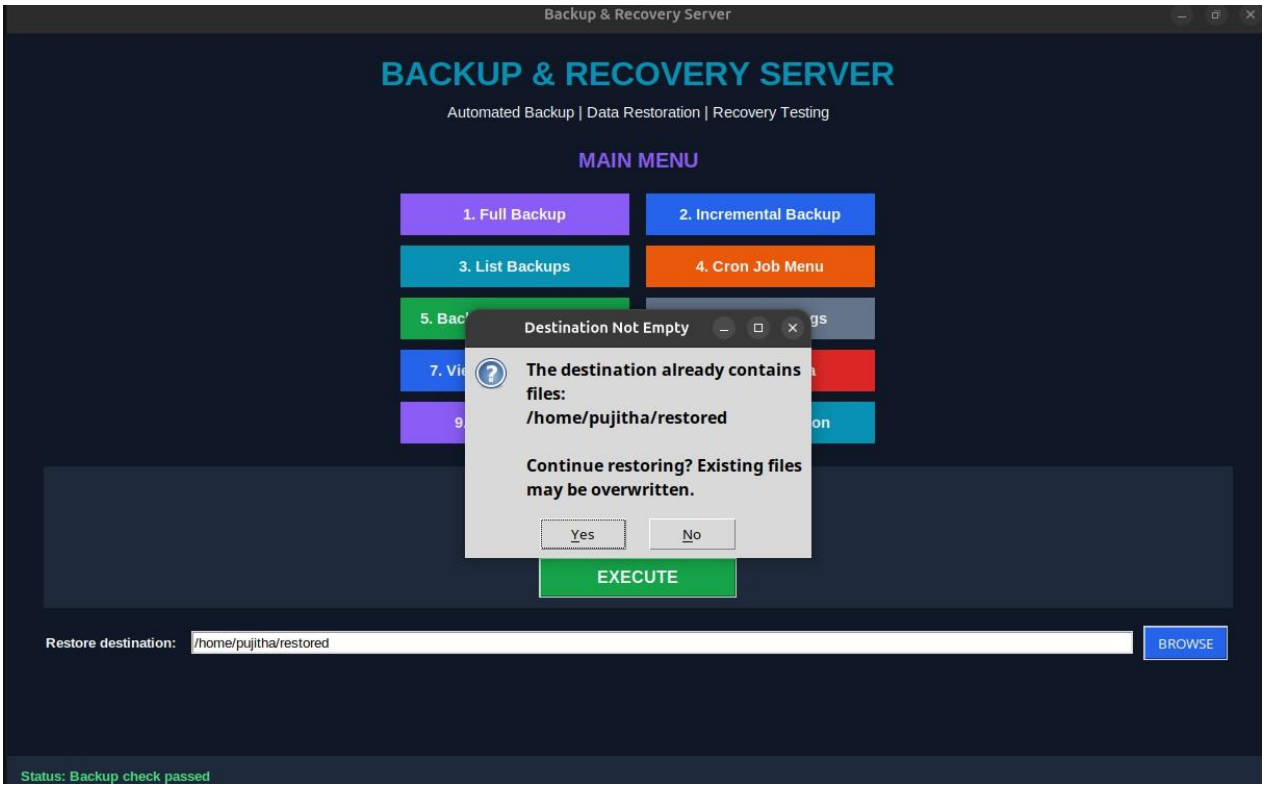


Figure 3.24: Data Restoration – Destination Not Empty Warning

This screenshot shows the dialog titled “Destination Not Empty”. It states that the destination already contains files (/home/pujitha/restored) and asks “Continue restoring? Existing files may be overwritten.” with Yes and No buttons. This prompt protects existing data in the destination from being overwritten without the user’s knowledge.

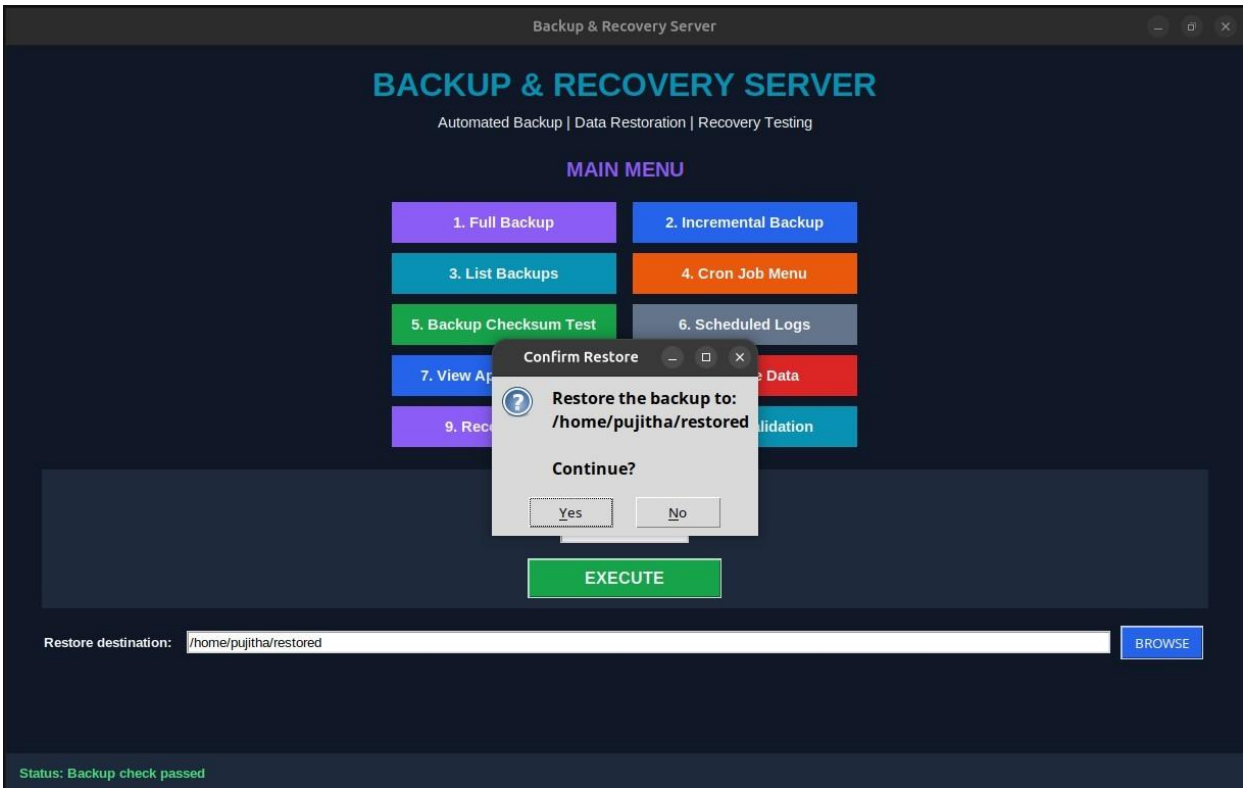


Figure 3.25: Data Restoration – Confirm Restore Prompt

The figure shows the "Confirm Restore" dialog. It displays the selected destination, /home/pujitha/restored, and asks the user to confirm with the question "Continue?". The user can choose Yes to proceed or No to cancel. Confirming the destination before the operation starts is important because restoration writes files to that location.

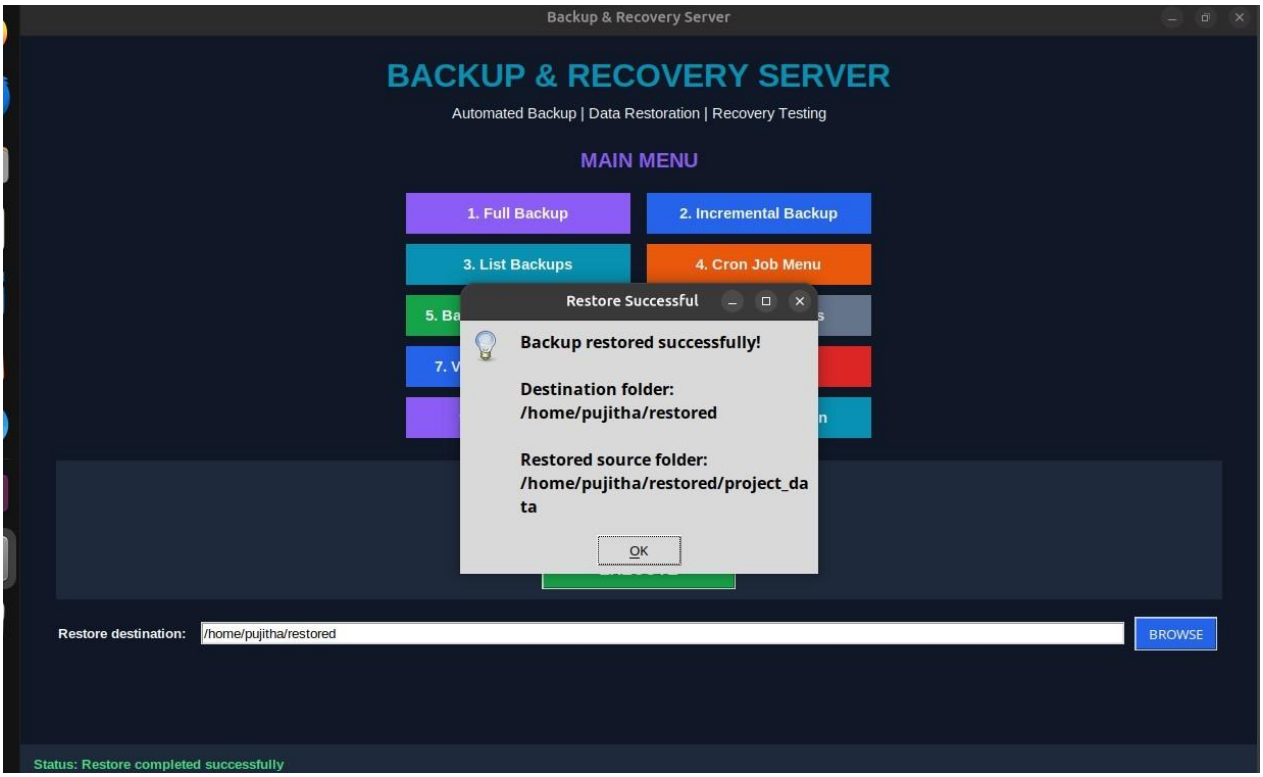


Figure 3.26: Data Restoration – Restore Successful Message

This screenshot shows the "Restore Successful" dialog, which reports "Backup restored successfully!". It also shows the destination folder (/home/pujitha/restored) and the restored source folder (/home/pujitha/restored/project_data). From this message, the restored data is placed in a project_data folder inside the chosen destination. The status bar displays "Status: Restore completed successfully".

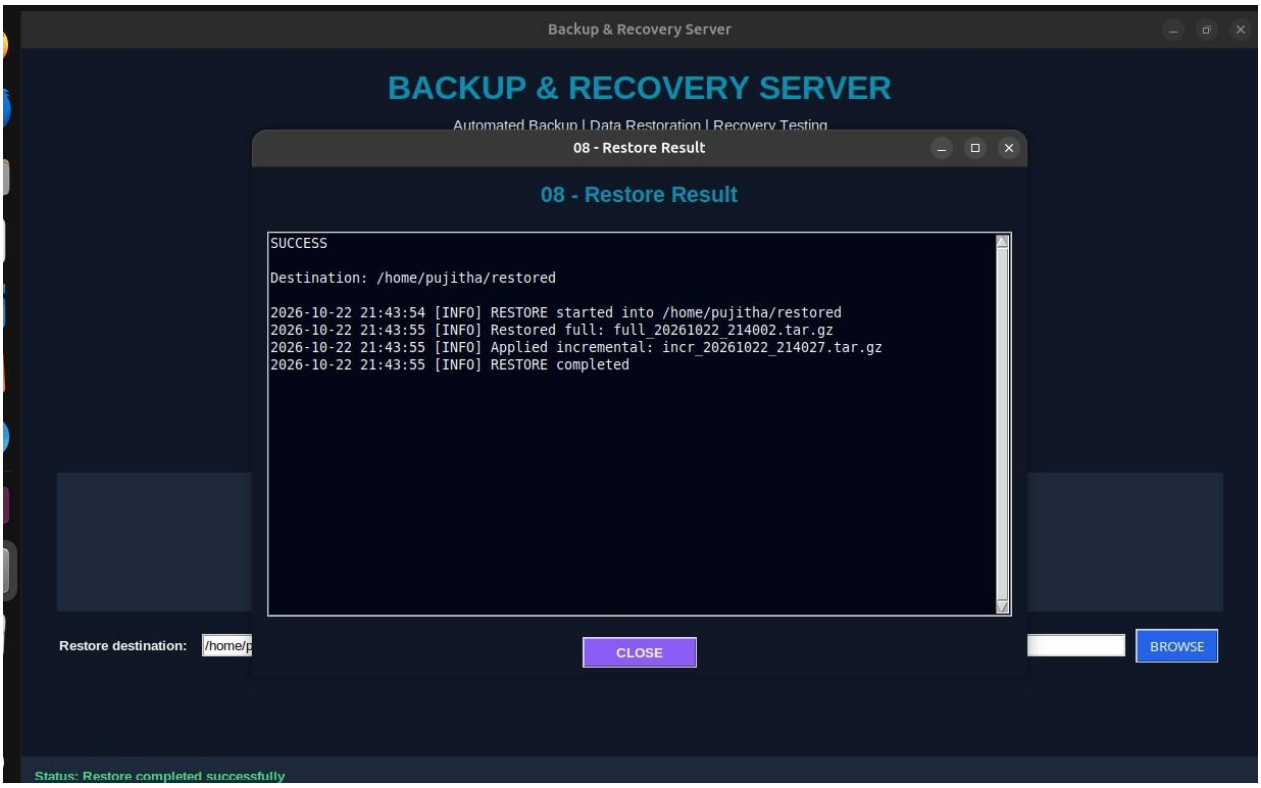


Figure 3.27: Data Restoration – Restore Result Log

The figure shows the "08 - Restore Result" window. It reports SUCCESS and the destination, followed by the log entries of the operation: the restore started at 2026-10-22 21:43:54, the full backup full_20261022_214002.tar.gz was restored, the incremental backup incr_20261022_214027.tar.gz was applied, and the restore completed. These two archives are the ones created by the full backup and incremental backup operations shown in Figures 3.17 and 3.18, which confirms that the restoration used the backups created earlier in the same session.

Result

The restoration function recovers data from the available full and incremental backups into the destination chosen by the user. Confirmation prompts, a success message and a detailed result window make the outcome of the operation visible. The screenshots show the messages produced by the application; the contents of the restored folder itself are not shown, and should be inspected in the destination when the restoration needs to be verified further.

3.2.11 RECOVERY TESTING

Recovery testing checks whether backup data can actually be used to recover files. A backup is only valuable if it can be restored, so a recovery test provides confidence that the stored archives are usable. A test should be performed in a destination separate from the real data, and its result should be inspected rather than relying only on a confirmation message.

In the Centralized Backup and Recovery Server, the recovery test is available as option 09 of the main menu. Before running it, the dashboard asks for confirmation with the question “Run the recovery test script now?”. When the user confirms, the dashboard runs a recovery-test script, and the result window displays the command that was executed (bash /home/pujitha/backup_system/scripts/recovery_test.sh), the exit code of the script, and the log lines written by the test.

The log lines show that the test is run in a separate directory whose name carries a random suffix (for example, /home/pujitha/recovery_test.xWFrY5), and not in the normal restore destination. The test ends with the message “RECOVERY TEST PASSED: restored data matches source”, which indicates that the data recovered from the backups was compared with the source data. This approach allows the recovery workflow to be exercised without touching the folder used for normal restoration.

Recovery Test – Implementation Output

The following screenshots show how the recovery test is started and the result that it displays.



Figure 3.28: Recovery Test – Confirmation Prompt

This screenshot shows the “Recovery Test” dialog, which asks “Run the recovery test script now?” with Yes and No buttons. The menu number 9 is visible in the input box of the dashboard behind the dialog, and the status bar still shows the message of the previous operation (“Restore completed successfully”).

The prompt gives the user an opportunity to cancel before the test begins.

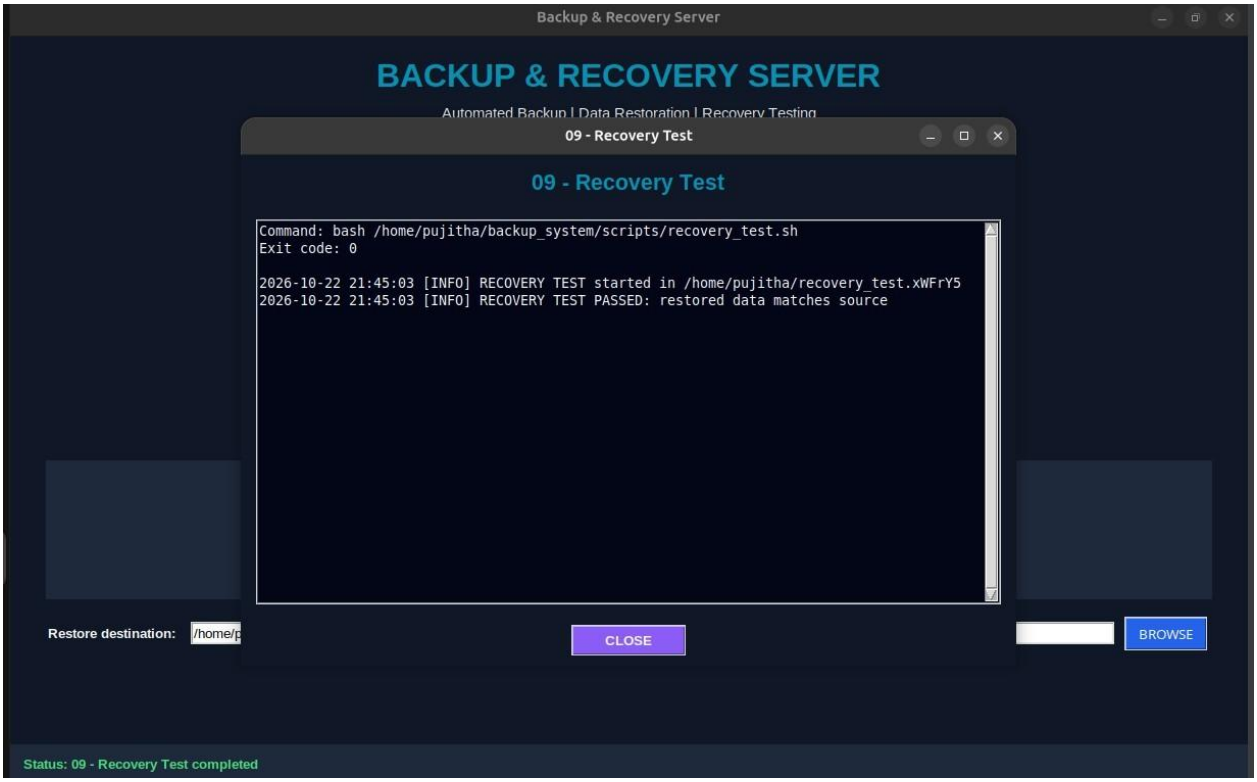


Figure 3.29: Recovery Test – Displayed Result

The figure shows the "09 - Recovery Test" window. It lists the command bash /home/pujitha/backup_system/scripts/recovery_test.sh with Exit code: 0, followed by two log entries dated 2026-10-22 21:45:03: the recovery test started in a temporary directory, and "RECOVERY TEST PASSED: restored data matches source". The status bar displays "Status: 09 - Recovery Test completed".

Result

The recovery test completed with exit code 0 and reported that the restored data matches the source. The same message also appears in the application log for the earlier tests shown in Figure 3.23. Together, these outputs demonstrate that the backups created by the system can be used to recover the project data in a separate test location.

3.2.12 INPUT VALIDATION AND ERROR HANDLING

Input validation checks the values supplied by the user before an operation is performed. It prevents unsupported choices from being treated as valid commands and provides feedback when the user enters an invalid value. In a backup and recovery system this is important because an incorrect selection could start an unintended operation.

The dashboard includes an Input Validation window that demonstrates this behaviour. It asks the user to enter a menu number from 1 to 10 and has a Validate Input button. When the entered value is outside the accepted range, the window displays a red message explaining why the value was rejected. A line of test examples (blank, abc, -1, 0, 1, 10, 11 and 1.5) is displayed below the button to suggest the kinds of input that can be tried: empty input, non-numeric text, negative numbers, zero, boundary values, out-of-range values and decimal numbers.

Error handling is also supported by the other output windows of the dashboard. The backup and recovery-test windows display the command that was run together with its exit code, and the status bar reports the most recent outcome. In the captured screenshots the exit code is 0, which indicates that the commands completed without error. Screenshots of failed operations are not included in the supplied material, so their messages are not described here.

Input Validation – Implementation Output

The following screenshot shows the Input Validation window after an invalid number has been entered.

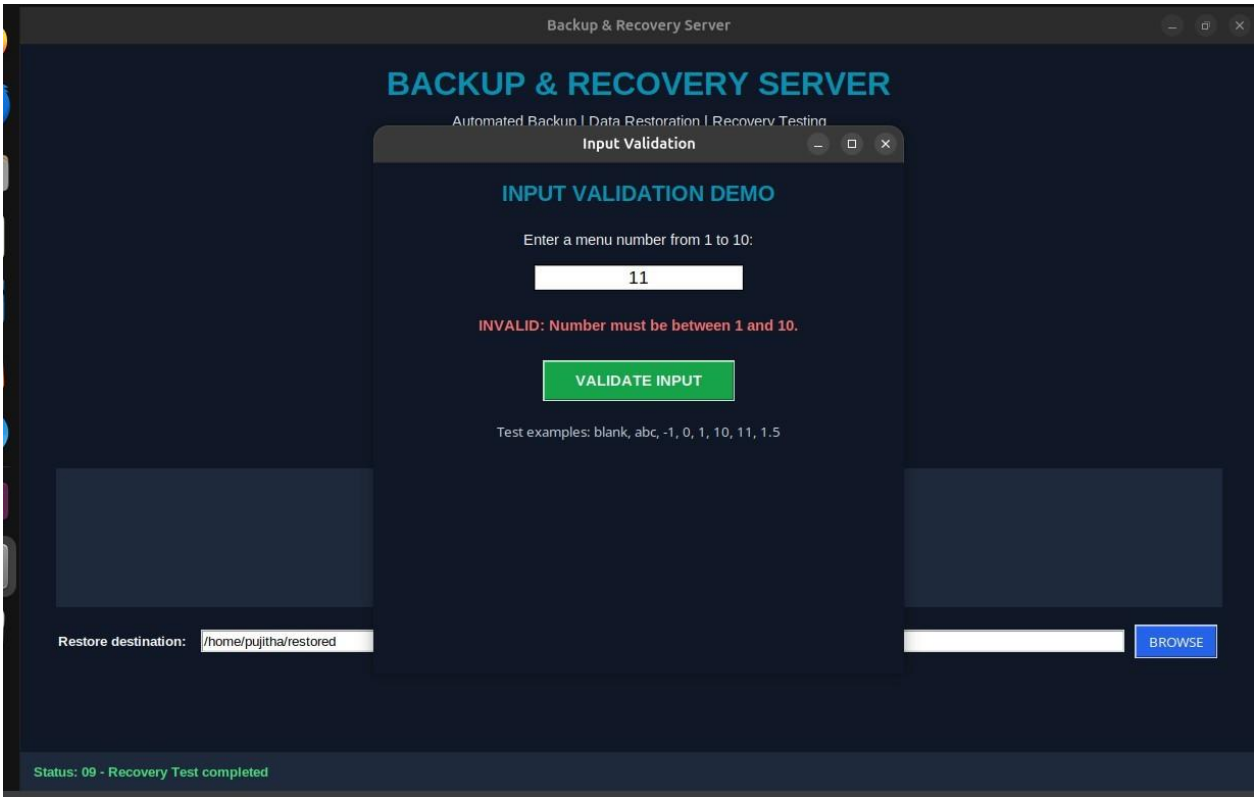


Figure 3.30: Input Validation Output

The screenshot shows the “INPUT VALIDATION DEMO” window. The prompt reads “Enter a menu number from 1 to 10:”, the value 11 has been entered, and the application displays the message “INVALID: Number must be between 1 and 10.” This message identifies the problem and tells the user which range is acceptable. The result for the other test examples listed in the window is not captured in this screenshot.

Result

The input validation function rejects a value outside the permitted range and explains the reason to the user. This prevents unsupported menu numbers from being accepted and guides the user toward a valid entry.

4. RESULTS AND TESTING

This chapter presents the results obtained from the implementation described in Chapter 3. The results are based on the source-code screenshots and the output screenshots of the dashboard. Each function of the Centralized Backup and Recovery Server is reviewed against the output that it produced, and the evidence available for each function is stated. A function is recorded as passed only when the corresponding screenshot clearly confirms the expected behaviour; where a screenshot shows only that a menu or window is available, this is stated explicitly.

4.1 IMPLEMENTATION RESULTS

The implementation provides a graphical dashboard titled “Backup & Recovery Server” with the sub-title “Automated Backup | Data Restoration | Recovery Testing”. The main menu contains numbered options for the backup, scheduling, logging, restoration, recovery-testing and validation functions. The results of each function are summarised in Table 4.1.

Table 4.1: Summary of Implemented Functions and Observed Outputs

Function	Dashboard option / window	Figure	Observed output
Full backup	01 - Full Backup	3.17	Command: backup.sh full; exit code 0; “full backup OK: full_20261022_214002.tar.gz”.
Incremental backup	02 - Incremental Backup	3.18	Command: backup.sh incr; exit code 0; “incr backup OK: incr_20261022_214027.tar.gz”.
List backups	03 - List Backups	3.19	“BACKUP INVENTORY” listing twelve full backups (0.3 KB each) followed by the incremental backups (0.2–0.3 KB each).
Cron job menu	Cron Job Menu	3.20	Four options: View Cron Configuration, Check Cron Service, View Scheduled Job Logs and Run Incremental Backup Now.
Backup execution / checksum test	05 - Backup Checksum Test	3.21	Each listed full and incremental archive is reported as “OK”; status bar shows “Backup check passed”.
Scheduled logs	Scheduled Job Logs	3.22	Exit code 0 and Cron log entries dated Oct 20 showing hourly runs of /etc/cron.hourly.
Application log	07 - Application Log	3.23	Time-stamped [INFO] entries for backups, restores and recovery tests, including “RECOVERY TEST PASSED”.
Data restoration	08 - Restore Result	3.24–3.27	Warning and confirmation prompts; “Backup restored successfully!”; result log listing the full and incremental archives restored.
Recovery test	09 - Recovery Test	3.28–3.29	Confirmation prompt; exit code 0; “RECOVERY TEST PASSED: restored data matches source”.
Input validation	Input Validation	3.30	Input 11 rejected with “INVALID: Number must be between 1 and 10.”

The outputs show that the backup operations create archives that follow a clear naming pattern: full backups are named full_YYYYMMDD_HHMMSS.tar.gz and incremental backups are named incr_YYYYMMDD_HHMMSS.tar.gz, where the date and time are those of the moment the backup was created. This naming makes it easy to identify the type and creation time of each archive in the backup listing, the checksum test and the restore log.

The backup inventory in Figure 3.19 and the checksum test in Figure 3.21 show that archives from more than one day (2026-10-20 and 2026-10-22) are stored by the system. The restore operation uses the full backup and the incremental backup created in the same session (Figures 3.17, 3.18 and 3.27), and the recovery test confirms that data recovered from the backups matches the source (Figures 3.23 and 3.29).

4.2 TESTING AND VALIDATION

The testing performed on the project was functional testing. Each function of the dashboard was executed, and its output was captured as a screenshot. The expected result of each test case was defined in advance, and the captured output was then compared with it. Source code screenshots were also captured for the backend and frontend components. Table 4.2 lists the test cases, the expected results, the evidence observed in the screenshots and the resulting status.

Table 4.2: Test Cases, Expected Results and Observed Evidence

Test case	Expected result	Observed evidence	Status
Backend and interface code	Source code is available for the interface and the processing logic	Figures 3.2–3.16 show the code sections	Code evidence
Full backup	A full backup runs and reports its result	Figure 3.17: exit code 0 and “full backup OK” message	Pass
Incremental backup	An incremental backup runs and reports its result	Figure 3.18: exit code 0 and “incr backup OK” message	Pass
List backups	Available backup files are displayed	Figure 3.19: inventory of full and incremental archives	Pass
Cron job menu	Scheduling-related controls are available	Figure 3.20: Cron Job Management menu with four options	Menu available; Cron schedule itself not verified
Backup checksum test	Archives are checked and the result is displayed	Figure 3.21: each listed archive reported as “OK”	Pass
Scheduled logs	Scheduled-activity records can be viewed	Figure 3.22: Cron log entries displayed	Logs viewable; execution of the backup job not shown
Application logs	Application log records can be viewed	Figure 3.23: time-stamped entries for backups, restores and tests	Pass
Data restoration	The restore workflow runs and reports its result	Figures 3.24–3.27: prompts, success message and result log	Pass; restored files not shown

Test case	Expected result	Observed evidence	Status
Recovery test	The recovery test runs and displays a result	Figures 3.28–3.29: exit code 0 and "RECOVERY TEST PASSED"	Pass
Input validation	An unsupported input receives an error message	Figure 3.30: input 11 rejected with an explanatory message	Pass for input 11 only

Testing Summary

The screenshots document the source code, the dashboard, the backup operations, the backup listing, the checksum test, the scheduling-related menu, the log views, the restoration workflow, the recovery test and the input validation. Of the eleven test cases, the backup, listing, checksum, application-log, restoration and recovery-test functions produced outputs that confirm the expected result. The remaining cases are recorded with the qualification shown in Table 4.2 because the screenshots establish that a menu, a log view or a message is available but do not by themselves prove the wider behaviour.

Observations and Limitations

The following points should be kept in mind when reading the test results:

- The Cron Job Menu confirms that scheduling controls are available. The Cron entries visible in the scheduled logs belong to the hourly system job (/etc/cron.hourly); they do not by themselves show that a scheduled backup job has been configured or executed. The Cron configuration should be checked separately using the View Cron Configuration and Check Cron Service options.
- The restoration screenshots show the messages and the log produced by the restore operation. The files inside the restored folder (/home/pujitha/restored/project_data) are not shown, so their contents should be inspected directly when a stronger verification is required.
- Only the input value 11 is shown in the input-validation screenshot. The results for the other listed examples (blank, abc, -1, 0, 1, 10 and 1.5) are not captured and are therefore not reported.
- The supplied material does not include timing or performance measurements, so no performance figures are reported. The archive sizes visible in the backup listing (0.2–0.3 KB) reflect the small test data set used in the project.

For stronger validation, the generated backup files should be confirmed in the backup directory, the relevant log entries should be inspected, and the restored files should be compared with the source data in the selected destination.

5. CONCLUSION AND FUTURE SCOPE

5.1 CONCLUSION

The Centralized Backup and Recovery Server demonstrates how Linux system administration concepts can be organised into a complete backup and recovery workflow. The project brings together a graphical dashboard, Bash scripts for backup and restoration, full and incremental backups, backup listing, checksum verification, Cron-based scheduling support, log viewing, data restoration, recovery testing and input validation.

The output screenshots document the user-facing workflow of the system. Full and incremental backups were created and reported as successful, the stored archives were listed and passed the checksum test, the application log recorded the operations performed, a restore was carried out with confirmation prompts and a result log, and the recovery test reported that the restored data matched the source. The input validation window rejected an out-of-range menu number with a clear message.

Through this project, the team gained practical experience with Linux commands, Bash scripting, archive handling, scheduling with Cron, logging, error handling and data recovery procedures. The project also showed the importance of testing recovery and not only creating backups, since the value of a backup depends on whether the data can be restored. Verifying the actual backup files, the Cron configuration and the restored data remains important to confirm that the system performs as expected in every environment where it is used.

5.2 FUTURE SCOPE

The project can be extended in several directions. The following enhancements are possible; they are suggestions for future work and are not described as features of the present implementation:

- Clearer backup-status reporting, with a summary of the outcome of each backup shown on the dashboard.
- Automatic verification of each generated archive immediately after it is created, so that integrity checking does not depend on the user running the checksum test.
- Stronger and more specific error messages for failed operations, with guidance on how to correct the problem.
- Configurable backup schedules that can be set from the dashboard instead of being edited through the Cron configuration.
- Backup-retention policies that remove or archive old backups automatically, since the backup directory grows with every backup that is created.
- A detailed history of successful and failed operations that can be searched and filtered.
- A disk-space check before a backup is started, so that a backup does not fail because the destination is full.
- A verification report after recovery that lists the restored files and compares them with the source data.

Any extension should be implemented and tested before it is described as an existing feature of the system.

REFERENCES

The following documentation sources describe the tools and technologies used in the project:

[1] Free Software Foundation, “Bash Reference Manual,” GNU Project. Available: https://www.gnu.org/software/bash/manual/

[2] Free Software Foundation, “GNU tar: an archiver tool,” GNU Project. Available: https://www.gnu.org/software/tar/manual/

[3] Free Software Foundation, “GNU Coreutils Manual” (sha256sum), GNU Project. Available: https://www.gnu.org/software/coreutils/manual/

[4] The Linux man-pages project, “crontab(5) – files used to schedule the execution of programs.” Available: https://man7.org/linux/man-pages/man5/crontab.5.html

[5] Python Software Foundation, “tkinter – Python interface to Tcl/Tk,” Python Documentation. Available: https://docs.python.org/3/library/tkinter.html

[6] Ubuntu Documentation, Canonical Ltd. Available: https://help.ubuntu.com/

APPENDICES

APPENDIX A: OUTPUT SCREENSHOT INDEX

The screenshots in this report are arranged in implementation order. Figure 3.1 shows the system architecture; Figures 3.2–3.6 cover the backend code; Figures 3.7–3.16 cover the frontend code; Figures 3.17–3.23 cover the backup operations, scheduling and logs; Figures 3.24–3.27 cover restoration; Figures 3.28–3.29 cover recovery testing; and Figure 3.30 covers input validation. Table A.1 gives the complete index.

Table A.1: Index of Figures

Figure	Title	Section
3.1	System architecture of the Backup & Recovery Server	3.1
3.2	Backend source code – configuration and setup	3.2.1
3.3	Backend source code – backup processing	3.2.1
3.4	Backend source code – file and backup handling	3.2.1
3.5	Backend source code – restoration and supporting functions	3.2.1
3.6	Backend source code – remaining processing logic	3.2.1
3.7	Frontend source code – interface layout	3.2.2
3.8	Frontend source code – controls and display area	3.2.2
3.9	Frontend source code – operation controls	3.2.2
3.10	Frontend source code – event handling	3.2.2
3.11	Frontend source code – output handling	3.2.2
3.12	Frontend source code – backup-related controls	3.2.2
3.13	Frontend source code – interface components	3.2.2
3.14	Frontend source code – operation wiring	3.2.2
3.15	Frontend source code – messages and interaction	3.2.2
3.16	Frontend source code – completed interface implementation	3.2.2
3.17	Full Backup Operation Output	3.2.3
3.18	Incremental Backup Operation Output	3.2.4
3.19	Listing Available Backup Files	3.2.5
3.20	Cron Job Menu for Scheduled Backup Management	3.2.6
3.21	Execution of the Backup Operation (Backup Checksum Test output)	3.2.7
3.22	Scheduled Backup Logs Display	3.2.8
3.23	Application Log Output	3.2.9
3.24	Data Restoration – Destination Not Empty Warning	3.2.10
3.25	Data Restoration – Confirm Restore Prompt	3.2.10
3.26	Data Restoration – Restore Successful Message	3.2.10
3.27	Data Restoration – Restore Result Log	3.2.10

Figure	Title	Section
3.28	Recovery Test – Confirmation Prompt	3.2.11
3.29	Recovery Test – Displayed Result	3.2.11
3.30	Input Validation Output	3.2.12

APPENDIX B: MAIN MENU OPTIONS OF THE DASHBOARD

Table B.1 lists the options of the main menu as they appear in the dashboard screenshots. Options 1 to 6 are fully visible in Figures 3.25 and 3.28. The labels of options 7 to 10 are partly hidden behind dialogs in those figures, so they are identified here from the titles of the windows that the options open.

Table B.1: Main Menu Options

Option	Function	Window title / label seen in the screenshots
1	Full backup	“1. Full Backup”; window “01 - Full Backup”
2	Incremental backup	“2. Incremental Backup”; window “02 - Incremental Backup”
3	List backups	“3. List Backups”; window “03 - List Backups”
4	Cron job menu	“4. Cron Job Menu”; window “Cron Job Menu”
5	Backup checksum test	“5. Backup Checksum Test”; window “05 - Backup Checksum Test”
6	Scheduled logs	“6. Scheduled Logs”; window “Scheduled Job Logs”
7	Application log	Window “07 - Application Log”
8	Restore data	Window “08 - Restore Result”
9	Recovery test	Window “09 - Recovery Test”
10	Input validation	Window “Input Validation”